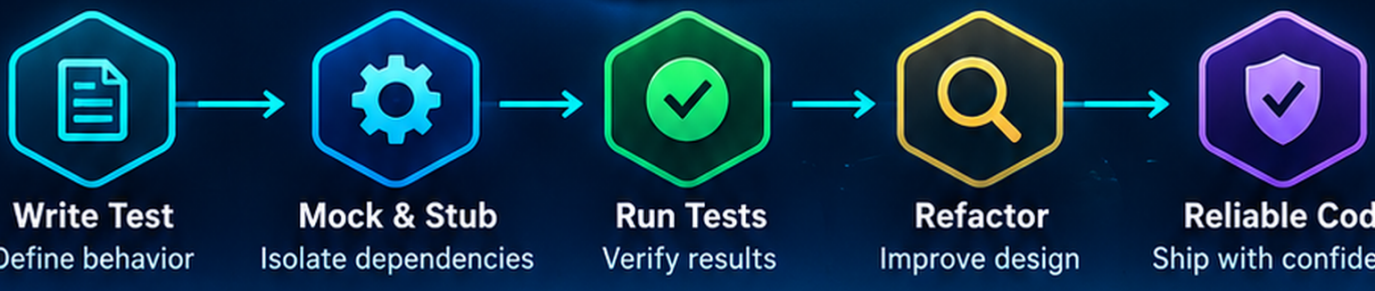
JUNIT + MOCKITO ZERO TO HERO INTERVIEW GUIDE

Java 17 • JUnit Jupiter 5 • Mockito 5

Clear tests. Reliable code. Confident interviews.





100 QUESTIONS | 100 FOLLOW-UPS | 12 LABS | 30-DAY PLAN

Write tests that prove behavior.

100 original interview questions, each with a beginner-friendly answer and a specific follow-up answer. The book focuses on JUnit Jupiter 5 and Mockito 5 using Java 17 examples.



HOW TO USE EVERY STUDY PAGE

1. Say the short answer aloud. **2.** Follow the diagram. **3.** Read the example. **4.** Explain the pitfall. **5.** Answer the follow-up.

VERIFICATION SCOPE

The 12 labs are provided as a Maven project. The JUnit/Mockito framework suite was not executed in this workspace because Maven and its dependencies were unavailable. The included README explains how to run the tests before using the material commercially.

Concepts: JUnit Jupiter 5.13.4 / Mockito 5.24.0 / Java 17. JUnit 6 also exists; this guide deliberately uses the JUnit 5 programming model to match common Spring Boot 3.x projects.

NAVIGATION / 20 CHAPTERS

Contents

Chapters are grouped in five-question interview sprints. Each chapter has a visual concept map and five complete Q&A pages.

01	Testing fundamentals	Q001-Q005
02	JUnit annotations and assertions	Q006-Q010
03	Fixtures and lifecycle	Q011-Q015
04	Parameterized testing	Q016-Q020
05	Errors, futures and time	Q021-Q025
06	JUnit Platform and extensions	Q026-Q030
07	Mockito object setup	Q031-Q035
08	Mockito stubbing patterns	Q036-Q040
09	Matchers and captors	Q041-Q045
10	Behavior verification	Q046-Q050

Appendix: 12 Java coding labs / quick comparison sheets / 30-day study route / reference links.

NAVIGATION / 20 CHAPTERS

Contents

Chapters are grouped in five-question interview sprints. Each chapter has a visual concept map and five complete Q&A pages.

11	Spies and special cases	Q051-Q055
12	Strictness and test isolation	Q056-Q060
13	Fakes, contracts and concurrency	Q061-Q065
14	Async and Spring MVC	Q066-Q070
15	Spring Data and security	Q071-Q075
16	Test strategy and coverage	Q076-Q080
17	Integration, containers and regressions	Q081-Q085
18	Refactoring and performance tests	Q086-Q090
19	Release readiness	Q091-Q095
20	Architect scenarios	Q096-Q100

Appendix: 12 Java coding labs / quick comparison sheets / 30-day study route / reference links.

THE TOOLKIT

JUnit and Mockito: how they fit

JUnit runs the test and checks expectations. Mockito replaces selected collaborators so the test can control external outcomes.



JUNIT IS RESPONSIBLE FOR

Test discovery, lifecycle, assertions, parameterized invocations, extensions, tags and reporting.

MOCKITO IS RESPONSIBLE FOR

Creating doubles, defining collaborator responses, capturing arguments and checking required interactions.

NEITHER LIBRARY PROVES

Database isolation, broker delivery, HTTP security or transaction durability just by mocking those dependencies. Use a focused integration test for the real boundary.

Design rule: Mock only what is outside the unit. Keep domain calculations and result values real.

Testing fundamentals

Choose the right test boundary and assert a visible requirement.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
Test input -> real unit -> expected outcome
```

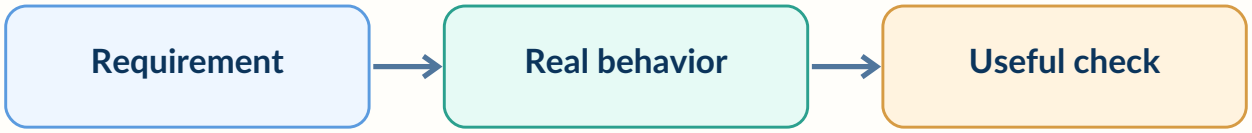
Five distinct questions are followed by five individual answers to deeper interview follow-ups.

What is a unit test?

IN ONE CLEAR ANSWER

A unit test checks one small behavior with controlled inputs and clear expected results. It should run quickly and isolate the reason for a failure; the unit may be one class or a small cluster of related classes.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
Test input -> real unit -> expected outcome
```

INTERVIEW FOLLOW-UP

Does a unit test have to test one method?

No. Test the visible behavior of a unit, including the calls needed to reach it. Splitting a workflow into artificial method-level tests can make refactoring harder.

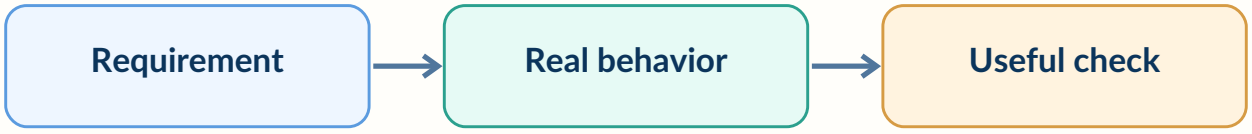
Why this matters: Choose the right test boundary and assert a visible requirement.

How are unit, integration and end-to-end tests different?

IN ONE CLEAR ANSWER

A unit test checks a narrow behavior with fast controlled dependencies. An integration test checks how real components work together, and an end-to-end test checks a complete user flow across boundaries.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
Test input -> real unit -> expected outcome
```

INTERVIEW FOLLOW-UP

Where should a repository query be tested?

Use a real test database for the query, indexes and mapping. Mocking a repository in a service unit test does not validate its SQL or ORM behavior.

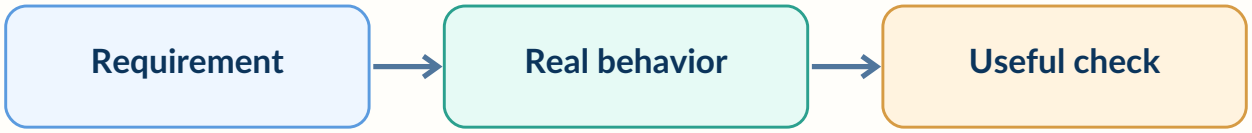
Why this matters: Choose the right test boundary and assert a visible requirement.

What makes a test reliable and maintainable?

IN ONE CLEAR ANSWER

A reliable test has one clear purpose, deterministic setup, observable assertions and useful failure messages. It avoids shared state, sleeps and assumptions about execution order.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
Test input -> real unit -> expected outcome
```

INTERVIEW FOLLOW-UP

Why should a test fail when production behavior changes?

Because tests are executable requirements. A test that always passes or asserts only a mock call can miss a broken business result.

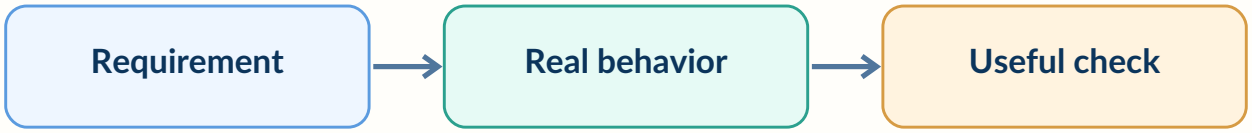
Why this matters: Choose the right test boundary and assert a visible requirement.

What is the Arrange-Act-Assert pattern?

IN ONE CLEAR ANSWER

Arrange creates inputs and collaborators; Act invokes the behavior once; Assert verifies its result and important side effects. This readable flow reduces accidental confusion between setup and expectations.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
Test input -> real unit -> expected outcome
```

INTERVIEW FOLLOW-UP

Can a test contain more than one assertion?

Yes, when assertions describe one coherent outcome. Split unrelated behaviors into different tests so a failure has a clear meaning.

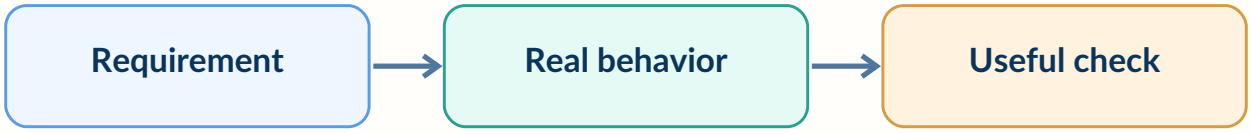
Why this matters: Choose the right test boundary and assert a visible requirement.

What is the difference between a test double, stub, mock, fake and spy?

IN ONE CLEAR ANSWER

A double replaces a collaborator. A stub returns prepared data, a mock records interactions, a fake has a working simplified implementation, and a spy wraps real behavior while allowing observation or stubbing.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
Test input -> real unit -> expected outcome
```

INTERVIEW FOLLOW-UP

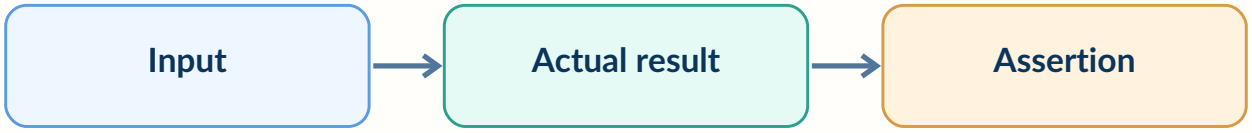
When is an in-memory fake better than a mock?

When realistic state transitions are essential and a simple deterministic fake can model them. Validate real database semantics separately.

Why this matters: Choose the right test boundary and assert a visible requirement.

JUnit annotations and assertions

Use the correct assertion for equality, identity, collections and absence.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
assertEquals(220, service.total(200, 10));
```

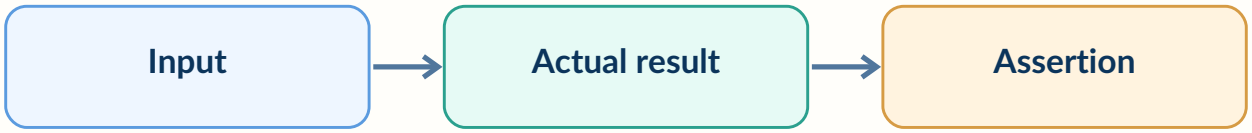
Five distinct questions are followed by five individual answers to deeper interview follow-ups.

What does @Test do in JUnit Jupiter?

IN ONE CLEAR ANSWER

@Test marks a method as a test executed by the Jupiter engine through the JUnit Platform. The method normally returns void and is not a private method.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(220, service.total(200, 10));
```

INTERVIEW FOLLOW-UP

Does a Java main method run JUnit tests automatically?

No. Use a JUnit-aware IDE or build-tool test runner with Jupiter engine available on the test runtime classpath.

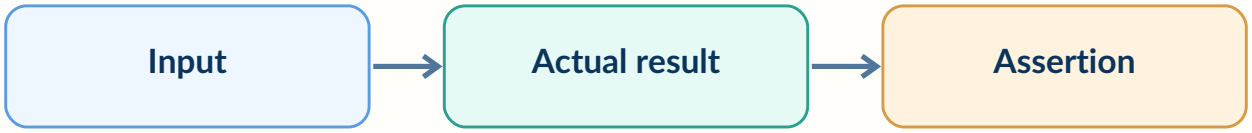
Why this matters: Use the correct assertion for equality, identity, collections and absence.

When should you use assertEquals instead of assertSame?

IN ONE CLEAR ANSWER

assertEquals tests equality under equals; assertSame requires the identical object reference. For domain values, equality is usually the intended contract.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(220, service.total(200, 10));
```

INTERVIEW FOLLOW-UP

Can two different objects pass assertEquals?

Yes, if equals returns true. Their references remain different, so assertSame would fail.

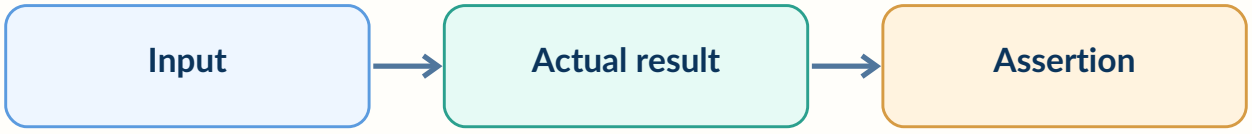
Why this matters: Use the correct assertion for equality, identity, collections and absence.

What do assertTrue, assertFalse and assertNull verify?

IN ONE CLEAR ANSWER

They verify boolean conditions or whether a reference is null. Prefer expressive assertions that show expected and actual values rather than vague compound conditions.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(220, service.total(200, 10));
```

INTERVIEW FOLLOW-UP

Why can assertEquals be easier to debug than assertTrue?

JUnit can show expected and actual values directly. A failed true/false expression may hide which value was wrong.

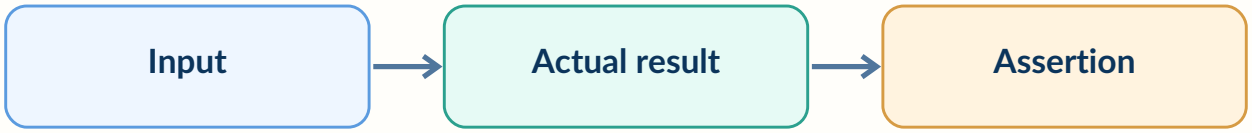
Why this matters: Use the correct assertion for equality, identity, collections and absence.

How does assertAll help?

IN ONE CLEAR ANSWER

assertAll evaluates multiple grouped assertions and reports all their failures together. It helps validate related fields in one returned value without stopping at the first failed field.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(220, service.total(200, 10));
```

INTERVIEW FOLLOW-UP

Should assertAll group unrelated workflows?

No. Keep a group focused on one outcome; unrelated checks should be in independent test methods.

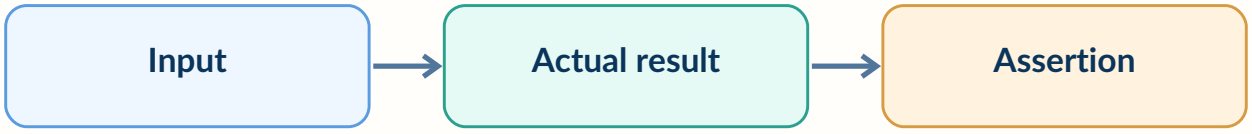
Why this matters: Use the correct assertion for equality, identity, collections and absence.

How do you test collections without depending on accidental order?

IN ONE CLEAR ANSWER

Choose an assertion matching the contract: exact ordered list when order is promised, or membership/set equality when it is not. Never assume hash map iteration order without a stated requirement.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(220, service.total(200, 10));
```

INTERVIEW FOLLOW-UP

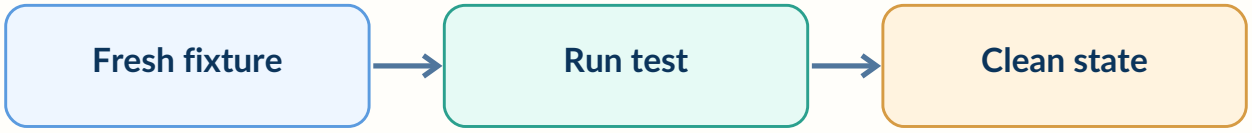
When is a sorted collection assertion wrong?

When sorting is not the product behavior; the test may hide a defect in required ordering or unnecessarily impose new behavior.

Why this matters: Use the correct assertion for equality, identity, collections and absence.

Fixtures and lifecycle

Make test setup predictable without relying on order.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
@BeforeEach void setup() { items = new ArrayList<>(); }
```

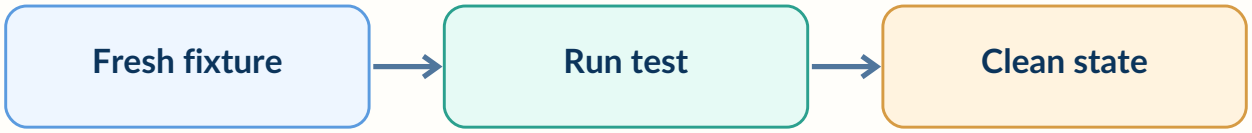
Five distinct questions are followed by five individual answers to deeper interview follow-ups.

What do @BeforeEach and @AfterEach do?

IN ONE CLEAR ANSWER

@BeforeEach runs setup before each test and @AfterEach runs cleanup after it. Keep fixture creation local enough that readers can understand it; cleanup must not hide failures.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@BeforeEach void setup() { items = new ArrayList<>(); }
```

INTERVIEW FOLLOW-UP

Is one @BeforeEach invocation shared by all tests?

No, it runs for each test invocation, including parameterized invocations, subject to the chosen test-instance lifecycle.

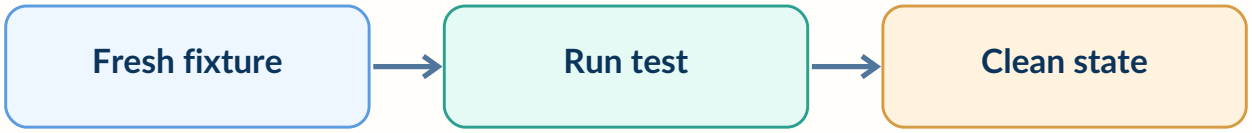
Why this matters: Make test setup predictable without relying on order.

What do @BeforeAll and @AfterAll do?

IN ONE CLEAR ANSWER

They surround all tests in a class, often to create and release expensive shared resources. By default they are static because Jupiter creates a new test instance per method.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@BeforeEach void setup() { items = new ArrayList<>(); }
```

INTERVIEW FOLLOW-UP

Can @BeforeAll be non-static?

Yes, when using @TestInstance(PER_CLASS). Ensure shared mutable state does not cause order-dependent tests.

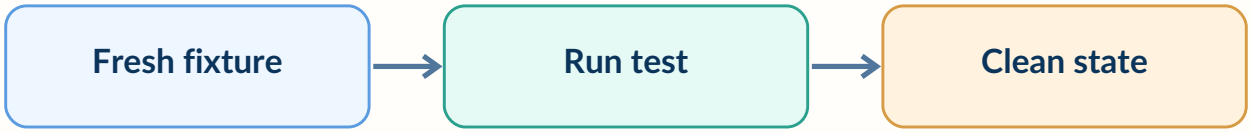
Why this matters: Make test setup predictable without relying on order.

What is the default JUnit Jupiter test-instance lifecycle?

IN ONE CLEAR ANSWER

The default is PER_METHOD: each test gets a fresh test-class instance. This helps isolate instance fields but does not reset static state, files or external services.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@BeforeEach void setup() { items = new ArrayList<>(); }
```

INTERVIEW FOLLOW-UP

Does PER_METHOD prevent all state leaks?

No. Static maps, caches, database records and background workers can still leak state between tests.

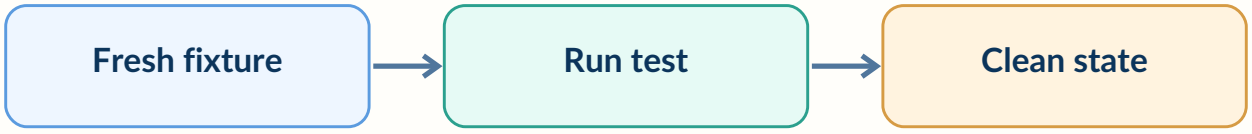
Why this matters: Make test setup predictable without relying on order.

When should @TestInstance(PER_CLASS) be used?

IN ONE CLEAR ANSWER

Use PER_CLASS when one test instance is intentional, such as expensive fixture ownership or non-static @BeforeAll. Document and synchronize mutable shared fields if parallel execution is enabled.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@BeforeEach void setup() { items = new ArrayList<>(); }
```

INTERVIEW FOLLOW-UP

What is the key risk of PER_CLASS?

One test can change fields observed by another. Tests may pass separately but fail as a suite.

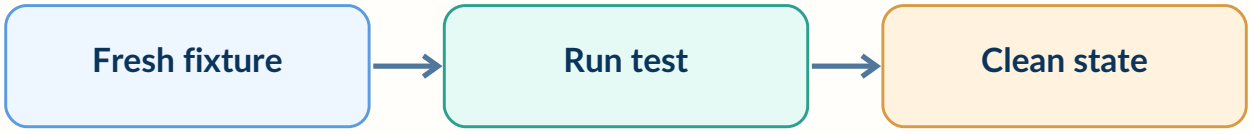
Why this matters: Make test setup predictable without relying on order.

Why should tests not depend on execution order?

IN ONE CLEAR ANSWER

Order-coupled tests hide shared state and fail under parallelism, retries or individual execution. Give each test its own setup and teardown or reset external state deliberately.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@BeforeEach void setup() { items = new ArrayList<>(); }
```

INTERVIEW FOLLOW-UP

Does @TestMethodOrder fix shared-state defects?

No. It only controls execution order; it does not make inter-test dependency a good design.

Why this matters: Make test setup predictable without relying on order.

Parameterized testing

Repeat one business rule over boundary inputs.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
@CsvSource({"17,false", "18,true", "19,true"})
```

Five distinct questions are followed by five individual answers to deeper interview follow-ups.

How does @ParameterizedTest work?

IN ONE CLEAR ANSWER

One test method runs for multiple supplied argument sets. Use it for the same rule over boundary inputs, not as a substitute for tests with different purposes.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@CsvSource({"17,false", "18,true", "19,true"})
```

INTERVIEW FOLLOW-UP

What happens when one argument row fails?

That invocation fails and is reported individually; other invocations can still run unless a runner stops the suite.

Why this matters: Repeat one business rule over boundary inputs.

How do @ValueSource, @CsvSource and @MethodSource differ?

IN ONE CLEAR ANSWER

@ValueSource supplies simple values, @CsvSource provides rows of text-converted fields, and @MethodSource can supply richer objects and Streams of Arguments.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@CsvSource({"17,false", "18,true", "19,true"})
```

INTERVIEW FOLLOW-UP

When is @MethodSource a better choice?

Use it for custom objects, nested lists, explicit nulls or complicated expected outcomes that are hard to express as CSV strings.

Why this matters: Repeat one business rule over boundary inputs.

How can you test null, empty and blank strings?

IN ONE CLEAR ANSWER

Provide separate inputs because null, empty and whitespace-only strings are different values. Assertions should reflect exactly what the contract permits or rejects.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@CsvSource({"17,false", "18,true", "19,true"})
```

INTERVIEW FOLLOW-UP

Can @ValueSource(strings={""}) provide null?

No. Use @NullSource or @NullAndEmptySource for null/empty combinations as appropriate.

Why this matters: Repeat one business rule over boundary inputs.

Why use @EnumSource in parameterized testing?

IN ONE CLEAR ANSWER

It enumerates a stable set of enum constants without hard-coded string conversions. Filter included or excluded names when only certain states follow the same rule.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@CsvSource({"17,false", "18,true", "19,true"})
```

INTERVIEW FOLLOW-UP

How do you catch a future new enum constant?

Test all values using @EnumSource without restrictive names, or use an exhaustive domain switch where available.

Why this matters: Repeat one business rule over boundary inputs.

When should you prefer a dynamic test or @TestFactory?

IN ONE CLEAR ANSWER

Dynamic tests are useful when the cases are created at runtime and names can be generated. They have different lifecycle semantics from independent @Test methods.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@CsvSource({"17,false", "18,true", "19,true"})
```

INTERVIEW FOLLOW-UP

Does @BeforeEach run before each dynamic child?

No. It runs around the factory method, not separately around every generated DynamicTest executable.

Why this matters: Repeat one business rule over boundary inputs.

Errors, futures and time

Test exceptional paths and bounded completion.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
assertThrows(IllegalArgumentException.class, () -> divide(5, 0));
```

Five distinct questions are followed by five individual answers to deeper interview follow-ups.

How do you assert an exception with assertThrows?

IN ONE CLEAR ANSWER

Wrap only the operation expected to fail, then assert the exception type and meaningful fields/message. assertThrows accepts the expected type or a subtype.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertThrows(IllegalArgumentException.class, () -> divide(5, 0));
```

INTERVIEW FOLLOW-UP

When would assertThrowsExactly be preferred?

When accepting a subclass would violate the contract. Otherwise assertThrows is often less brittle.

Why this matters: Test exceptional paths and bounded completion.

How do you test that no exception is thrown?

IN ONE CLEAR ANSWER

Use assertDoesNotThrow on the operation and inspect the returned value if useful. A test without assertions can still pass while doing nothing meaningful.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertThrows(IllegalArgumentException.class, () -> divide(5, 0));
```

INTERVIEW FOLLOW-UP

Does assertDoesNotThrow prove correctness?

No. Also check the resulting state, returned value or durable effect where the requirement demands it.

Why this matters: Test exceptional paths and bounded completion.

What is the difference between assertTimeout and assertTimeoutPreemptively?

IN ONE CLEAR ANSWER

assertTimeout executes on the calling thread and checks elapsed time afterward; preemptive timeout can execute work in another thread and interrupt it. Preemptive timeouts can disrupt thread-bound contexts.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertThrows(IllegalArgumentException.class, () -> divide(5, 0));
```

INTERVIEW FOLLOW-UP

Why is preemptive timeout risky with Spring transactions?

Work may execute outside the thread-bound test transaction, causing unexpected committed changes. Prefer a non-preemptive timeout or operation-level deadline.

Why this matters: Test exceptional paths and bounded completion.

How should asynchronous code be tested without Thread.sleep?

IN ONE CLEAR ANSWER

Coordinate with CompletableFuture, CountDownLatch or bounded await-style mechanisms and assert terminal outcomes. Use explicit deadlines so tests fail rather than hang indefinitely.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertThrows(IllegalArgumentException.class, () -> divide(5, 0));
```

INTERVIEW FOLLOW-UP

Does waiting for a future guarantee an external side effect completed?

Only if completion semantics promise that effect. Verify the actual observable state or result boundary.

Why this matters: *Test exceptional paths and bounded completion.*

What does assertIterableEquals check?

IN ONE CLEAR ANSWER

It compares iterables element by element in iteration order. For maps or order-insensitive collections, choose a contract-matching assertion instead.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertThrows(IllegalArgumentException.class, () -> divide(5, 0));
```

INTERVIEW FOLLOW-UP

Will two sets with equal elements always iterate in equal order?

No; their iteration behavior can differ. Compare set equality when order is not part of the contract.

Why this matters: Test exceptional paths and bounded completion.

JUnit Platform and extensions

Understand discovery, extensions, tags and safe parallel tests.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
@Tag("integration")
@Test void verifiesDatabaseContract() { ... }
```

Five distinct questions are followed by five individual answers to deeper interview follow-ups.

What are assumptions in JUnit?

IN ONE CLEAR ANSWER

Assumptions such as assumeTrue abort or skip a test when its required environment is unavailable, rather than reporting a failed assertion. Use sparingly so important checks are not silently skipped.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@Tag("integration")
@Test void verifiesDatabaseContract() { ... }
```

INTERVIEW FOLLOW-UP

Should a broken configuration become an assumption?

Usually no. Fail clearly if the test environment is required for the build, and reserve assumptions for intentionally optional conditions.

Why this matters: Understand discovery, extensions, tags and safe parallel tests.

What is the JUnit Platform and how does it relate to Jupiter?

IN ONE CLEAR ANSWER

The JUnit Platform discovers and launches test engines; Jupiter supplies modern annotations and execution rules. Vintage can execute older JUnit 3/4 tests when configured.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@Tag("integration")
@Test void verifiesDatabaseContract() { ... }
```

INTERVIEW FOLLOW-UP

Why can tests compile but not execute?

The test engine or build-tool provider may be missing or misconfigured even though the Jupiter API is present.

Why this matters: Understand discovery, extensions, tags and safe parallel tests.

What does @Nested improve?

IN ONE CLEAR ANSWER

@Nested groups tests by state or behavior under descriptive inner classes. It helps readers understand a workflow such as empty cart versus populated cart.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@Tag("integration")
@Test void verifiesDatabaseContract() { ... }
```

INTERVIEW FOLLOW-UP

Does @Nested create a separate production object automatically?

No. Arrange the fixture yourself and understand the outer and nested test-instance lifecycle.

Why this matters: Understand discovery, extensions, tags and safe parallel tests.

When should you use @DisplayName?

IN ONE CLEAR ANSWER

Use it when a business-readable test label explains behavior better than a method name. Good names state input, action and outcome without restating implementation details.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@Tag("integration")
@Test void verifiesDatabaseContract() { ... }
```

INTERVIEW FOLLOW-UP

How should a test name describe a bug regression?

Describe the formerly broken scenario and expected result, e.g., shouldNotChargeTwiceForSameRequestKey.

Why this matters: Understand discovery, extensions, tags and safe parallel tests.

How do tags and test suites help large projects?

IN ONE CLEAR ANSWER

@Tag can classify tests by purpose (unit, integration, slow); build tools and launchers can include or exclude tags. Keep fast feedback separate from slower integration coverage without hiding failures.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@Tag("integration")
@Test void verifiesDatabaseContract() { ... }
```

INTERVIEW FOLLOW-UP

Should all tests run on every commit?

Run a meaningful fast suite on each change and slower suites in CI gates; never permanently omit high-risk integration verification.

Why this matters: Understand discovery, extensions, tags and safe parallel tests.

Mockito object setup

Create mocks and keep the class under test real.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest { @Mock UserRepo repo; }
```

Five distinct questions are followed by five individual answers to deeper interview follow-ups.

What is a JUnit extension?

IN ONE CLEAR ANSWER

An extension hooks into test lifecycle, parameters, exceptions and other execution points. MockitoExtension is one example that manages Mockito mocks in Jupiter tests.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest { @Mock UserRepo repo; }
```

INTERVIEW FOLLOW-UP

When should an extension be chosen over @BeforeEach?

When reusable cross-cutting test behavior or parameter resolution is needed across classes, rather than ad hoc duplicated setup.

Why this matters: Create mocks and keep the class under test real.

Why must parallel test execution be designed deliberately?

IN ONE CLEAR ANSWER

Parallel tests may share static state, system properties, ports, files and external databases. Enable parallel execution only after fixtures, resource ownership and concurrency checks are safe.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest { @Mock UserRepo repo; }
```

INTERVIEW FOLLOW-UP

Can PER_METHOD alone make parallel database tests safe?

No. They may still modify the same external rows or schema; use isolation, unique data or serialization.

Why this matters: Create mocks and keep the class under test real.

What does Mockito.mock create?

IN ONE CLEAR ANSWER

It creates a test double implementing the requested type so you can control its answers and inspect interactions. By default an unstubbed method returns a default value, not domain truth.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest { @Mock UserRepo repo; }
```

INTERVIEW FOLLOW-UP

Should every class in a test be mocked?

No. Keep the class under test real; mock external collaborators only when isolation is useful.

Why this matters: Create mocks and keep the class under test real.

What do @Mock and @ExtendWith(MockitoExtension.class) provide?

IN ONE CLEAR ANSWER

@Mock declares a mock field or parameter; MockitoExtension creates/cleans up annotated mocks for JUnit Jupiter tests. Without initialization, a @Mock field can remain null.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest { @Mock UserRepo repo; }
```

INTERVIEW FOLLOW-UP

Is MockitoExtension a Spring extension?

No. It is Mockito's Jupiter integration and works without loading a Spring ApplicationContext.

Why this matters: Create mocks and keep the class under test real.

What does @InjectMocks actually do?

IN ONE CLEAR ANSWER

Mockito tries constructor, setter or field injection of available mocks into the class under test. It is not a general dependency injection container and may leave some collaborators unresolved.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest { @Mock UserRepo repo; }
```

INTERVIEW FOLLOW-UP

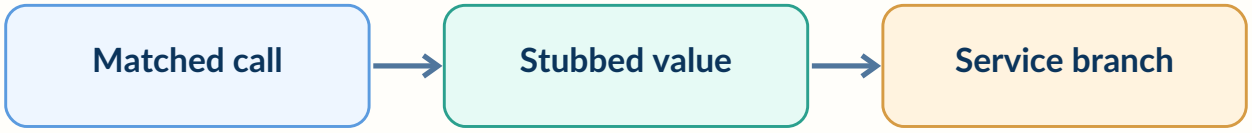
When is manual constructor injection better?

When explicit construction makes required dependencies and test wiring clearer, especially for immutable services.

Why this matters: Create mocks and keep the class under test real.

Mockito stubbing patterns

Return and throw controlled collaborator outcomes.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
when(repo.findName(7L)).thenReturn("Meena");
```

Five distinct questions are followed by five individual answers to deeper interview follow-ups.

How do mocks differ from spies?

IN ONE CLEAR ANSWER

A mock has no real behavior unless configured; a spy delegates to real methods by default. Spies can run side effects and require care during stubbing.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
when(repo.findName(7L)).thenReturn("Meena");
```

INTERVIEW FOLLOW-UP

Which is safer for a database repository test?

Neither validates SQL. Use a real repository against a test database and reserve mocks for caller-unit tests.

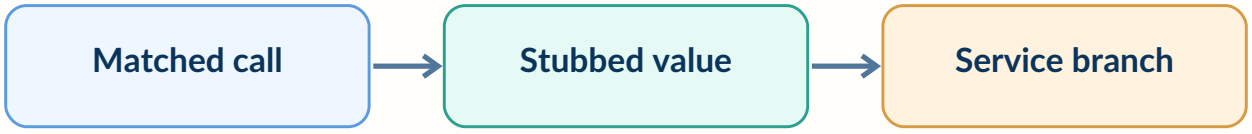
Why this matters: Return and throw controlled collaborator outcomes.

What are default answers for an unstubbed Mockito method?

IN ONE CLEAR ANSWER

Mockito returns type-appropriate defaults such as zero, false, empty optional/collection in supported cases, or null. Never treat a default as a correct business response.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
when(repo.findName(7L)).thenReturn("Meena");
```

INTERVIEW FOLLOW-UP

Why can an unstubbed call hide a defect?

A default false or empty result might match the test's expected outcome even though a necessary dependency call was omitted.

Why this matters: Return and throw controlled collaborator outcomes.

How does when(...).thenReturn(...) work?

IN ONE CLEAR ANSWER

It registers a response for a matching method invocation on a mock. When the production code later calls the method with matching arguments, the configured result is returned.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
when(repo.findName(7L)).thenReturn("Meena");
```

INTERVIEW FOLLOW-UP

Should you stub a method that your test does not call?

No. Unnecessary stubs confuse the test and strict stubbing may flag them.

Why this matters: Return and throw controlled collaborator outcomes.

When do doReturn(...).when(...) and when(...).thenReturn(...) differ?

IN ONE CLEAR ANSWER

For ordinary mocks either may work, but when(spy.method()) calls the real spy method during stubbing. doReturn(...).when(spy).method() avoids that real invocation.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
when(repo.findName(7L)).thenReturn("Meena");
```

INTERVIEW FOLLOW-UP

Why can a spy throw while being stubbed?

The real method runs inside when(spy.method()). Use doReturn when real execution is unsafe at setup time.

Why this matters: Return and throw controlled collaborator outcomes.

How do thenThrow and doThrow differ?

IN ONE CLEAR ANSWER

thenThrow configures a non-void stubbed call to throw; doThrow can stub a void method and can be used with spies. Choose checked exceptions permitted by the method signature.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
when(repo.findName(7L)).thenReturn("Meena");
```

INTERVIEW FOLLOW-UP

What failure should a payment gateway mock throw?

Model a domain-relevant timeout or rejection; the service must distinguish uncertain outcome from definite refusal.

Why this matters: Return and throw controlled collaborator outcomes.

Matchers and captors

Match only the needed inputs and capture important output objects.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
verify(publisher).publish(eventCaptor.capture());
```

Five distinct questions are followed by five individual answers to deeper interview follow-ups.

How do thenAnswer and Answer work?

IN ONE CLEAR ANSWER

An Answer computes a stub result from the invocation and its arguments. Use it sparingly for concise rules; excessive answers can reproduce the implementation under test.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(publisher).publish(eventCaptor.capture());
```

INTERVIEW FOLLOW-UP

When is thenAnswer better than many thenReturn calls?

When inputs determine a simple result rule and separate stubs would obscure that input-output relationship.

Why this matters: Match only the needed inputs and capture important output objects.

How does chained stubbing work for repeated calls?

IN ONE CLEAR ANSWER

Mockito supports consecutive answers, e.g., thenReturn(first, second), useful for retry tests. Once specified answers are exhausted, the last configured value is generally repeated.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(publisher).publish(eventCaptor.capture());
```

INTERVIEW FOLLOW-UP

How can retries be asserted safely?

Verify final behavior and the allowed attempt count; avoid relying on uncontrolled sleeps or nondeterministic scheduling.

Why this matters: Match only the needed inputs and capture important output objects.

What do lenient stubs mean?

IN ONE CLEAR ANSWER

Leniency relaxes specific strict-stubbing checks, which can help shared fixtures in narrow cases. Prefer moving stubs into tests that actually need them rather than disabling protection broadly.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(publisher).publish(eventCaptor.capture());
```

INTERVIEW FOLLOW-UP

Why is lenient() on every mock harmful?

It hides dead setup, argument mismatches and unused stubs, weakening tests during refactoring.

Why this matters: Match only the needed inputs and capture important output objects.

What are argument matchers like any() and eq()?

IN ONE CLEAR ANSWER

Matchers describe which invocation arguments should match a stub or verification. If you use matchers for any argument in one invocation, supply matchers for all arguments in that invocation.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(publisher).publish(eventCaptor.capture());
```

INTERVIEW FOLLOW-UP

Why does mixing eq(5) with a raw second value fail?

Mockito records matchers separately and expects their count to match method arguments. Use eq(...) for the other value too.

Why this matters: Match only the needed inputs and capture important output objects.

How does ArgumentCaptor work?

IN ONE CLEAR ANSWER

It captures values passed to a mocked collaborator so you can assert what the class under test sent. It is most helpful when emitted data is the observable contract.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(publisher).publish(eventCaptor.capture());
```

INTERVIEW FOLLOW-UP

When should you avoid ArgumentCaptor?

When a simple eq(expected) argument verification expresses the requirement more clearly.

Why this matters: Match only the needed inputs and capture important output objects.

Behavior verification

Verify critical effects without over-specifying implementation.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
verify(gateway, never()).charge(any());
```

Five distinct questions are followed by five individual answers to deeper interview follow-ups.

How should you verify a method was called once?

IN ONE CLEAR ANSWER

Use verify(mock).method(arguments), which defaults to times(1), then verify the observable result if relevant. A call check alone does not prove domain correctness.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(gateway, never()).charge(any());
```

INTERVIEW FOLLOW-UP

Is it good to verify every method call?

No. Over-verification couples tests to implementation details and makes safe refactoring expensive.

Why this matters: Verify critical effects without over-specifying implementation.

How do times, never and atLeast differ?

IN ONE CLEAR ANSWER

times(n) requires an exact invocation count, never() requires none, and atLeast(n) allows more. Choose a count only when it is part of the business or retry contract.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(gateway, never()).charge(any());
```

INTERVIEW FOLLOW-UP

When is never() a high-value assertion?

When preventing an unsafe action is the requirement, e.g., never charge a card after invalid input.

Why this matters: Verify critical effects without over-specifying implementation.

What does InOrder verify?

IN ONE CLEAR ANSWER

It verifies selected interactions occurred in a specific order across one or more mocks. Use it only if ordering changes correctness, such as save-before-publish intent.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(gateway, never()).charge(any());
```

INTERVIEW FOLLOW-UP

Does InOrder prove an external transaction committed?

No. It only observes calls to doubles; use an integration test to validate transaction behavior.

Why this matters: Verify critical effects without over-specifying implementation.

When should verifyNoMoreInteractions be used?

IN ONE CLEAR ANSWER

Use it when unexpected collaborator calls would violate an important boundary. Applying it everywhere makes tests fragile when harmless calls are added.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(gateway, never()).charge(any());
```

INTERVIEW FOLLOW-UP

What is a better assertion for side-effect prevention?

Verify that the dangerous operation was never invoked and that the returned failure/status is correct.

Why this matters: Verify critical effects without over-specifying implementation.

How do you verify a negative branch such as a rejected order?

IN ONE CLEAR ANSWER

Assert the business error and confirm no write or downstream charge occurred. Arrange the repository to return the rejection condition without building an unnecessary full application.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(gateway, never()).charge(any());
```

INTERVIEW FOLLOW-UP

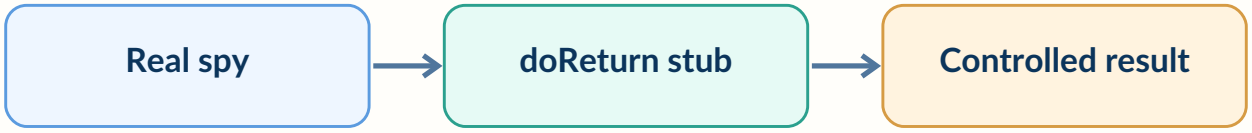
Would verifying only that save was never called be sufficient?

No. Also assert the outcome so a test cannot pass when the service silently does nothing.

Why this matters: Verify critical effects without over-specifying implementation.

Spies and special cases

Avoid accidentally executing real behavior during spy stubbing.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
doReturn("x").when(spy).read();
```

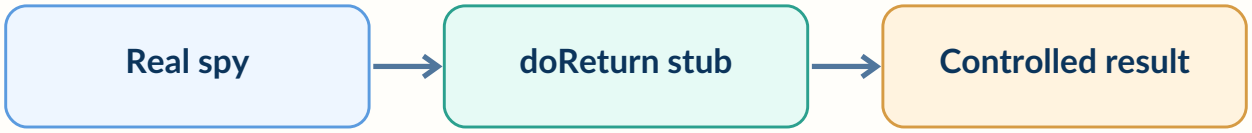
Five distinct questions are followed by five individual answers to deeper interview follow-ups.

What is a Mockito spy and what risks does it create?

IN ONE CLEAR ANSWER

A spy wraps a real object and invokes real methods unless configured otherwise. That can trigger network, file or mutable-state effects during testing and stubbing.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
doReturn("x").when(spy).read();
```

INTERVIEW FOLLOW-UP

Can a spy replace a missing integration test?

No. It may call some real logic but still does not prove that a database or network integration behaves correctly.

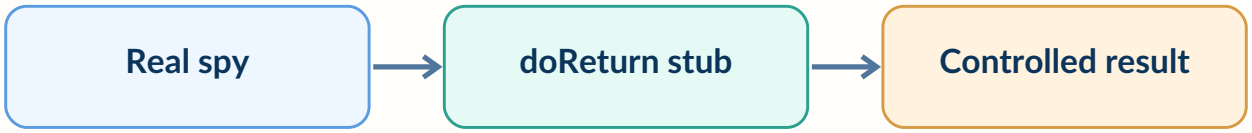
Why this matters: Avoid accidentally executing real behavior during spy stubbing.

How should you stub a spy safely?

IN ONE CLEAR ANSWER

Use doReturn, doThrow or doAnswer for methods whose real execution should not run during setup. Prefer a small real domain object without spying when possible.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
doReturn("x").when(spy).read();
```

INTERVIEW FOLLOW-UP

Why is doReturn safer than when on a spy?

when(spy.call()) evaluates call immediately; doReturn(...).when(spy).call() configures the stub without executing the real method.

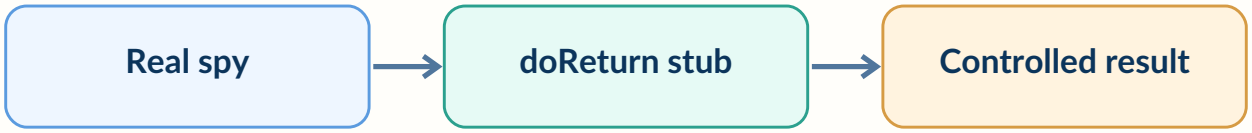
Why this matters: Avoid accidentally executing real behavior during spy stubbing.

How do you test a void method with Mockito?

IN ONE CLEAR ANSWER

Stub failures using doThrow(...).when(mock).voidMethod(), or verify the invocation using verify(mock).voidMethod(...). Most void effects should also have an observable contract.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
doReturn("x").when(spy).read();
```

INTERVIEW FOLLOW-UP

Should a void method test assert only interactions?

No. If the method changes state, writes data or publishes events, test the resulting state using a suitable real boundary where needed.

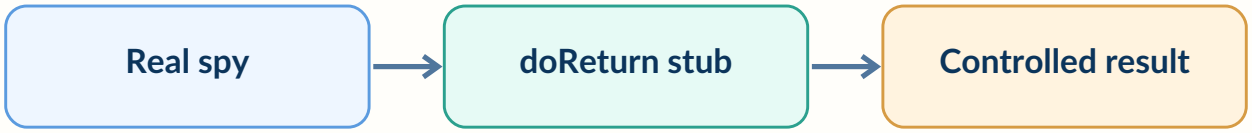
Why this matters: Avoid accidentally executing real behavior during spy stubbing.

When should static methods be mocked?

IN ONE CLEAR ANSWER

Mockito's scoped static mocking can isolate an unavoidable hard-coded static dependency. Prefer injecting Clock, adapters or suppliers instead of routine static mocking.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
doReturn("x").when(spy).read();
```

INTERVIEW FOLLOW-UP

Why must MockedStatic be closed?

The static mock is scoped and must be released to prevent interference with later tests, usually by try-with-resources.

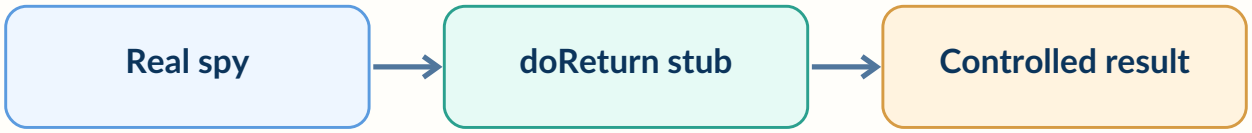
Why this matters: Avoid accidentally executing real behavior during spy stubbing.

Can Mockito mock final classes and methods?

IN ONE CLEAR ANSWER

Modern Mockito 5 uses inline mocking by default, enabling final types in supported JVM configurations. Mocking ability still depends on runtime instrumentation rules.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
doReturn("x").when(spy).read();
```

INTERVIEW FOLLOW-UP

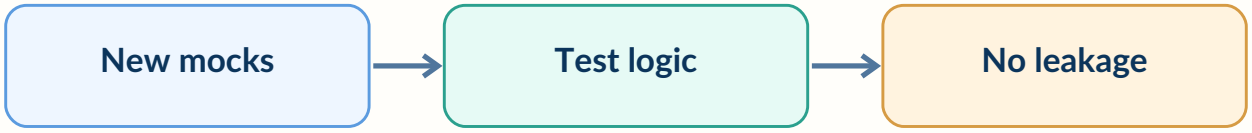
Should tests rely heavily on mocking domain records?

Usually no. Construct real immutable records and mock the external dependency that returns them; values are cheap and clarify expectations.

Why this matters: Avoid accidentally executing real behavior during spy stubbing.

Strictness and test isolation

Treat unused stubs and shared state as actionable signals.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
// MockitoExtension creates mocks for each test invocation.
```

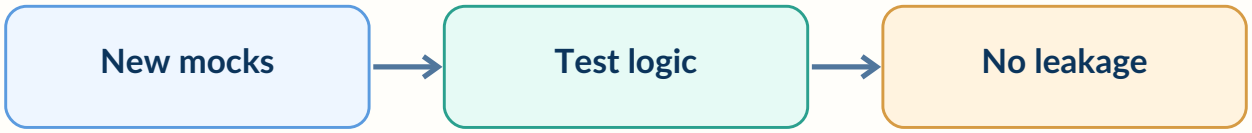
Five distinct questions are followed by five individual answers to deeper interview follow-ups.

What does deep stubbing do and why is it risky?

IN ONE CLEAR ANSWER

RETURNS_DEEP_STUBS builds mocks for chained return values. It can hide a poorly designed dependency chain and couples tests to internal navigation.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
// MockitoExtension creates mocks for each test invocation.
```

INTERVIEW FOLLOW-UP

What design change avoids a.b().c().d() mocking?

Expose a focused collaborator method that returns the required domain value, or compose the lower-level calls inside an adapter.

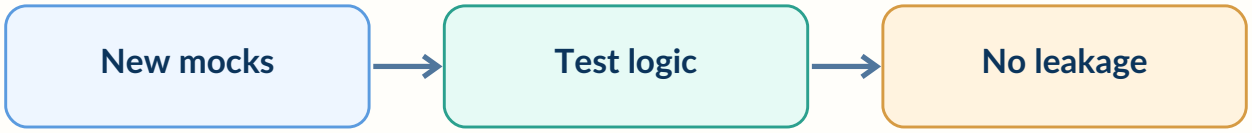
Why this matters: Treat unused stubs and shared state as actionable signals.

How do you test constructors that create hard-coded dependencies?

IN ONE CLEAR ANSWER

First prefer refactoring to constructor injection. Constructor mocking can be a temporary tool for legacy code, but adds instrumentation complexity and fragile tests.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
// MockitoExtension creates mocks for each test invocation.
```

INTERVIEW FOLLOW-UP

What is the most useful seam in legacy code?

Separate object construction from business logic so tests can inject a controlled collaborator without modifying global state.

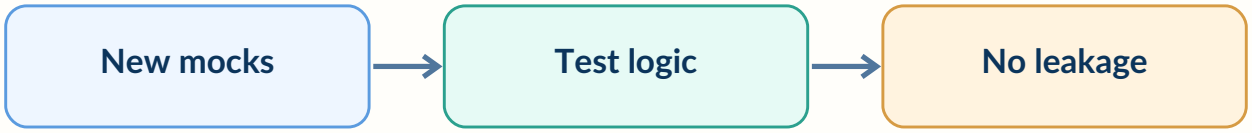
Why this matters: Treat unused stubs and shared state as actionable signals.

What is the purpose of strict stubbing in Mockito?

IN ONE CLEAR ANSWER

Strict stubbing points out unnecessary configured interactions and argument mismatches that often indicate stale or incorrect tests. Keep tests intentionally small.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
// MockitoExtension creates mocks for each test invocation.
```

INTERVIEW FOLLOW-UP

How should you fix an unnecessary stubbing exception?

Remove the unused stub or move it into the test that needs it, rather than globally disabling strictness.

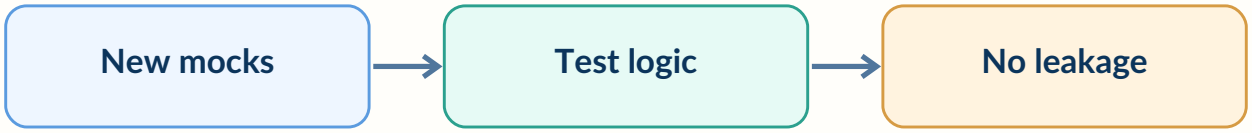
Why this matters: Treat unused stubs and shared state as actionable signals.

Why are shared mutable mocks across tests dangerous?

IN ONE CLEAR ANSWER

Tests can leak interactions, stubs and invocation history, especially under parallel execution. Prefer fresh mocks per test with MockitoExtension or explicit construction.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
// MockitoExtension creates mocks for each test invocation.
```

INTERVIEW FOLLOW-UP

Does reset(mock) solve all test isolation issues?

No. It only resets that mock's configuration and history, not external resources or hidden static state; fresh fixture objects are clearer.

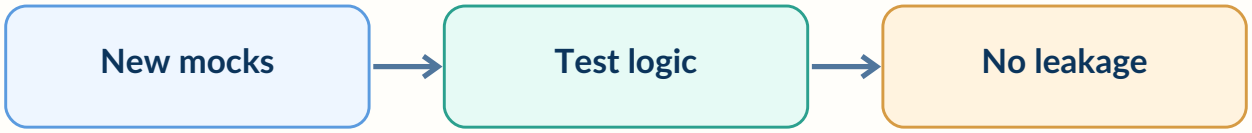
Why this matters: Treat unused stubs and shared state as actionable signals.

What is overmocking?

IN ONE CLEAR ANSWER

Overmocking means replacing too much real domain logic with scripted interaction, so tests verify the script rather than production behavior. Keep pure calculations and value objects real.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
// MockitoExtension creates mocks for each test invocation.
```

INTERVIEW FOLLOW-UP

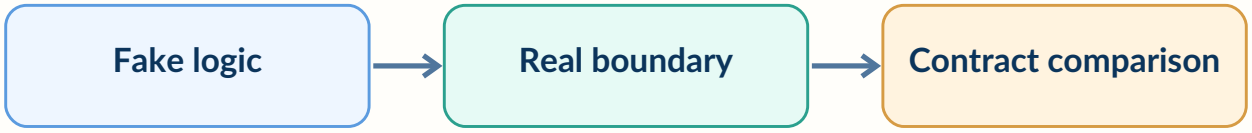
What symptom suggests overmocking?

A test must mock several methods of the class under test or knows the exact private call chain, yet checks no business result.

Why this matters: Treat unused stubs and shared state as actionable signals.

Fakes, contracts and concurrency

Recognize where mocks stop proving correctness.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
CountDownLatch ready = new CountDownLatch(2);
```

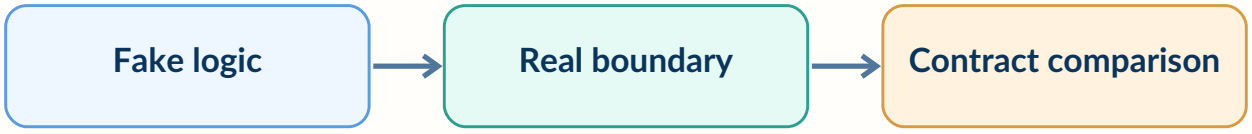
Five distinct questions are followed by five individual answers to deeper interview follow-ups.

What is a contract test for a fake implementation?

IN ONE CLEAR ANSWER

A contract test runs the same required behavior against the fake and real adapter to uncover semantic differences. It is especially helpful for edge cases and failure outcomes.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
CountDownLatch ready = new CountDownLatch(2);
```

INTERVIEW FOLLOW-UP

Can a HashMap fake enforce database uniqueness under concurrency?

Not by default. Even a thread-safe fake may differ from transactions, collation and SQL constraints; run real integration checks.

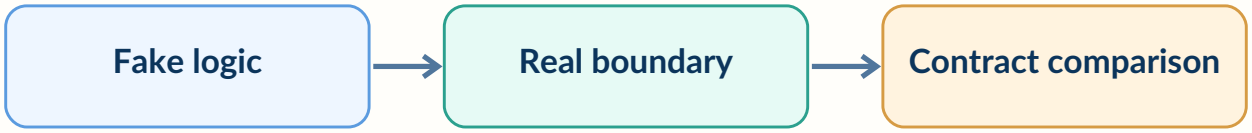
Why this matters: Recognize where mocks stop proving correctness.

When is an interaction test justified instead of a state test?

IN ONE CLEAR ANSWER

Use it when the interaction itself is a required effect, such as suppressing payment, sending exactly one audit event or passing a stable idempotency key.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
CountDownLatch ready = new CountDownLatch(2);
```

INTERVIEW FOLLOW-UP

Why not verify all repository reads?

Reads are often an implementation detail. Focus on outcome and required side effects, with integration tests for repository semantics.

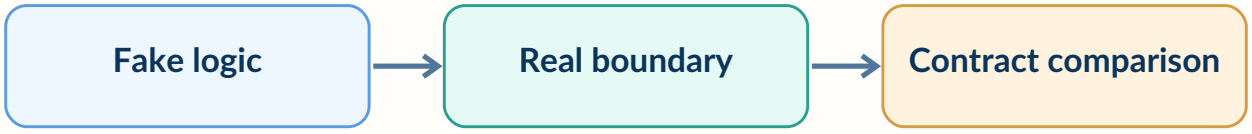
Why this matters: Recognize where mocks stop proving correctness.

How should you test CompletableFuture success and failure?

IN ONE CLEAR ANSWER

Provide already-completed futures or controlled CompletableFuture instances and assert final outcomes with bounded get/join or Jupiter timeout rules. Test both normal completion and exceptional completion.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
CountDownLatch ready = new CountDownLatch(2);
```

INTERVIEW FOLLOW-UP

What failure occurs if you never complete a test future?

The test can hang. Use controlled completion and a time bound with informative failure output.

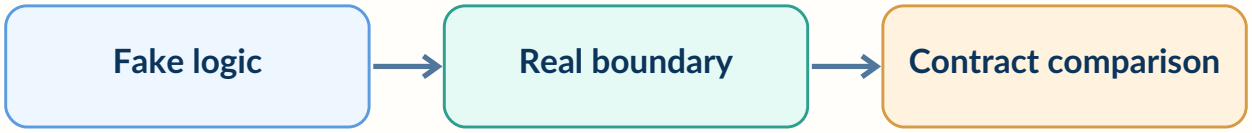
Why this matters: Recognize where mocks stop proving correctness.

How can you test scheduled retries without sleeping?

IN ONE CLEAR ANSWER

Inject a scheduler or clock and drive virtual time or scheduled tasks explicitly. Verify backoff decisions, maximum attempts and final outcome deterministically.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
CountDownLatch ready = new CountDownLatch(2);
```

INTERVIEW FOLLOW-UP

Does a fake clock replace a real retry integration test?

No. It validates policy; real networking and scheduler behavior still need an integration test.

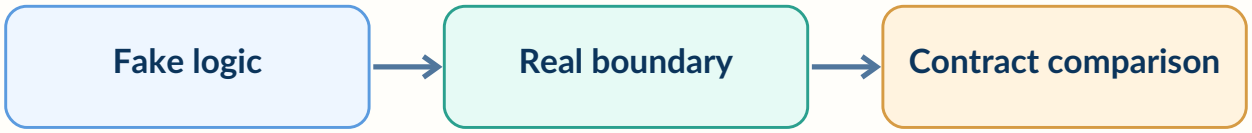
Why this matters: Recognize where mocks stop proving correctness.

How should race conditions be tested?

IN ONE CLEAR ANSWER

Use latches or barriers to coordinate the contested moment and repeat stress checks. Assert an invariant such as at most one successful stock reservation.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
CountDownLatch ready = new CountDownLatch(2);
```

INTERVIEW FOLLOW-UP

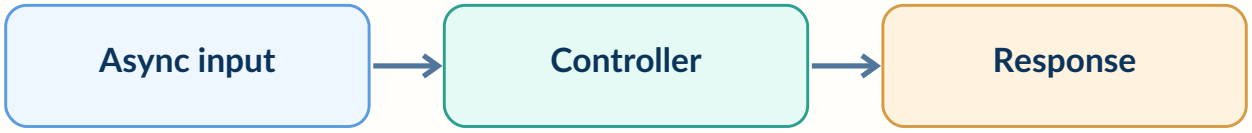
Can passing a concurrent test prove absence of every race?

No. Stress tests provide evidence only for the interleavings exercised; design invariants must be enforced by correct synchronization or atomic storage.

Why this matters: Recognize where mocks stop proving correctness.

Async and Spring MVC

Control async results and choose an appropriate Spring test slice.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
@WebMvcTest(OrderController.class)
class OrderControllerTest { /* MockMvc */ }
```

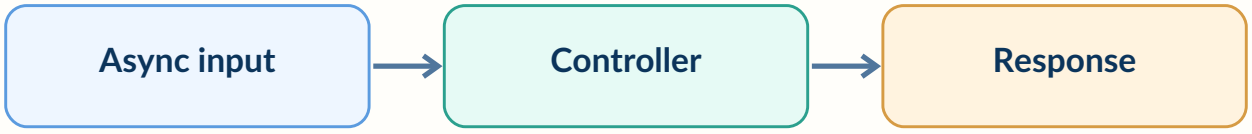
Five distinct questions are followed by five individual answers to deeper interview follow-ups.

What is the difference between timeout testing and cancellation testing?

IN ONE CLEAR ANSWER

Timeout testing checks the caller's deadline result; cancellation testing checks whether underlying work stops or releases resources. A timed-out future can leave a running external action.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@WebMvcTest(OrderController.class)
class OrderControllerTest { /* MockMvc */ }
```

INTERVIEW FOLLOW-UP

Should a timeout guarantee a payment was not processed?

No. Model the outcome as possibly unknown and reconcile using a stable idempotency key.

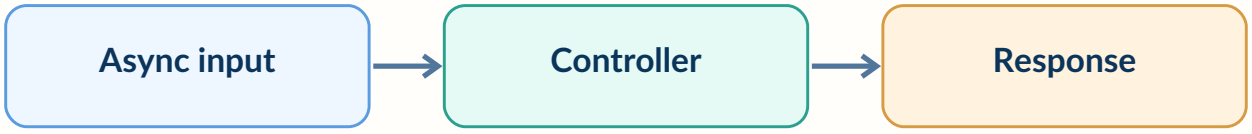
Why this matters: Control async results and choose an appropriate Spring test slice.

How do you test event consumption with at-least-once delivery?

IN ONE CLEAR ANSWER

Send the same event twice through a controlled harness and check that the business effect occurs once. A database integration test should prove the deduplication marker and effect commit together.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@WebMvcTest(OrderController.class)
class OrderControllerTest { /* MockMvc */ }
```

INTERVIEW FOLLOW-UP

Can verify(consumer, times(1)) prove exactly-once business effect?

No. It observes one mock invocation, not a durable transaction or broker redelivery behavior.

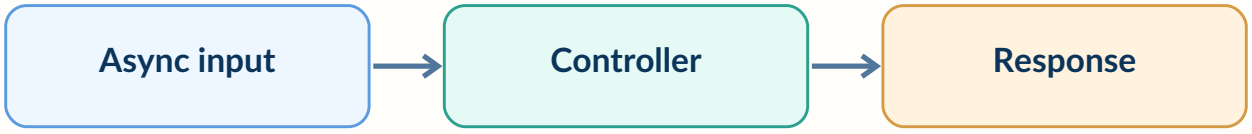
Why this matters: Control async results and choose an appropriate Spring test slice.

What is the difference between @SpringBootTest and a plain unit test?

IN ONE CLEAR ANSWER

@SpringBootTest starts a Spring application context to exercise integration wiring. A plain JUnit + Mockito test instantiates a service without Spring and is usually much faster.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@WebMvcTest(OrderController.class)
class OrderControllerTest { /* MockMvc */ }
```

INTERVIEW FOLLOW-UP

Should simple pricing logic use @SpringBootTest?

Usually not. Test pricing as a plain unit and reserve context tests for configuration, wiring and infrastructure behavior.

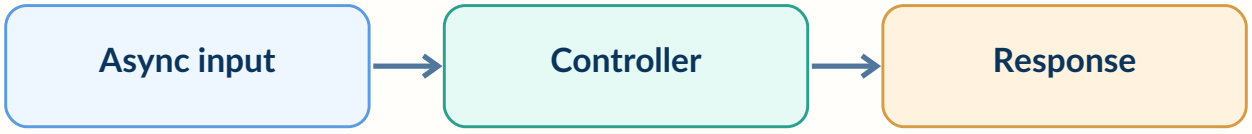
Why this matters: Control async results and choose an appropriate Spring test slice.

How does @WebMvcTest differ from @SpringBootTest?

IN ONE CLEAR ANSWER

@WebMvcTest focuses on MVC components and related infrastructure; @SpringBootTest loads broader application context. Use the smallest context that exercises the behavior being claimed.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@WebMvcTest(OrderController.class)
class OrderControllerTest { /* MockMvc */ }
```

INTERVIEW FOLLOW-UP

What is still needed to test real authentication?

Configure Spring Security filters and test identities, then test relevant authorized and forbidden paths.

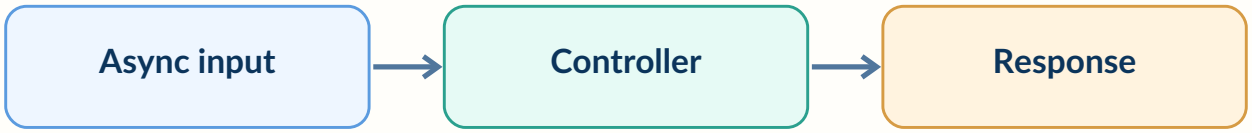
Why this matters: Control async results and choose an appropriate Spring test slice.

When should MockMvc be used?

IN ONE CLEAR ANSWER

MockMvc drives Spring MVC request handling without a real server socket, verifying routes, validation, serialization and response contracts.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@WebMvcTest(OrderController.class)
class OrderControllerTest { /* MockMvc */ }
```

INTERVIEW FOLLOW-UP

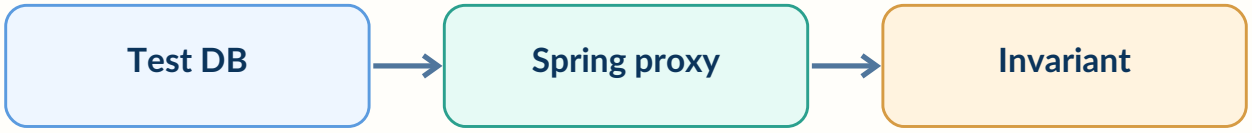
Does MockMvc prove a reverse proxy or TLS handshake works?

No. It does not open a real network socket. Use a running-server integration or end-to-end test for those boundaries.

Why this matters: Control async results and choose an appropriate Spring test slice.

Spring Data and security

Test ORM mappings, security rules and rollback in managed contexts.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
@DataJpaTest
class OrderRepositoryTest { /* real SQL */ }
```

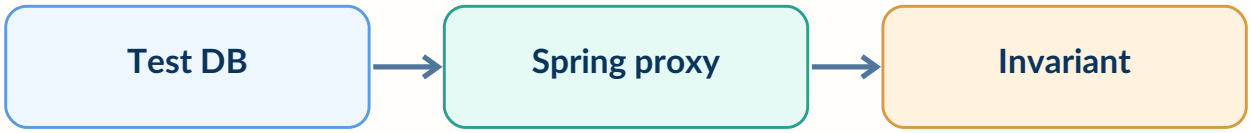
Five distinct questions are followed by five individual answers to deeper interview follow-ups.

What is the purpose of @DataJpaTest?

IN ONE CLEAR ANSWER

It configures a focused JPA test slice, often with transactional rollback after each test. Use it to test mappings, query methods, constraints and persistence behavior.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@DataJpaTest
class OrderRepositoryTest { /* real SQL */ }
```

INTERVIEW FOLLOW-UP

Why can a repository test pass on H2 but fail on PostgreSQL?

SQL syntax, types, collation, transaction semantics and query plans differ; use the target database, e.g., through Testcontainers, for important compatibility tests.

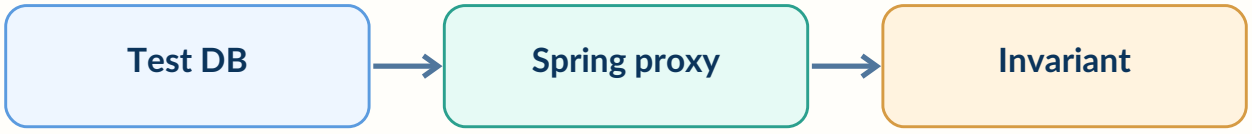
Why this matters: Test ORM mappings, security rules and rollback in managed contexts.

When should Spring dependencies be replaced by mocks in context tests?

IN ONE CLEAR ANSWER

Replace only a collaboration outside the slice being tested; Spring Boot's replacement annotation depends on Boot version. Prefer a plain unit test when no Spring behavior matters.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@DataJpaTest
class OrderRepositoryTest { /* real SQL */ }
```

INTERVIEW FOLLOW-UP

Is a context-level mock the same as MockitoExtension?

No. The former replaces a bean in a Spring ApplicationContext; MockitoExtension manages mocks for a plain Jupiter test.

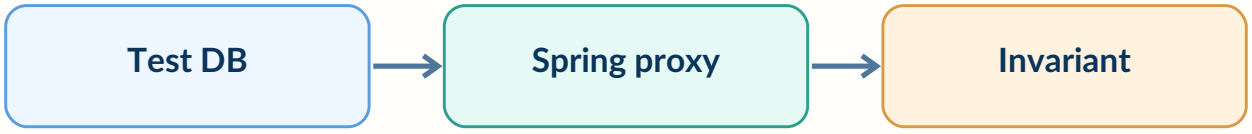
Why this matters: Test ORM mappings, security rules and rollback in managed contexts.

How do you test @Transactional behavior correctly?

IN ONE CLEAR ANSWER

Run an integration test that invokes the Spring-proxied service and checks database changes and rollback boundaries. A Mockito call check does not execute a real transaction.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@DataJpaTest
class OrderRepositoryTest { /* real SQL */ }
```

INTERVIEW FOLLOW-UP

Why can self-invocation break a transaction test?

Default proxy-based advice does not intercept calls made from one method to another on the same instance.

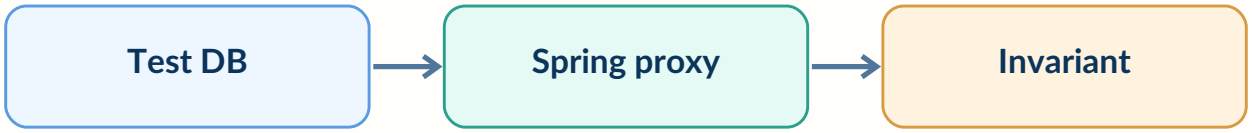
Why this matters: Test ORM mappings, security rules and rollback in managed contexts.

How do you test validation of REST input?

IN ONE CLEAR ANSWER

Send valid, missing, malformed and boundary requests through MVC and assert status plus structured errors. Separate Java bean-validation behavior from service-level business rules.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@DataJpaTest
class OrderRepositoryTest { /* real SQL */ }
```

INTERVIEW FOLLOW-UP

Should every invalid request invoke a mocked service?

No. For rejected input, verify that the controller or framework stops before performing a dangerous business operation.

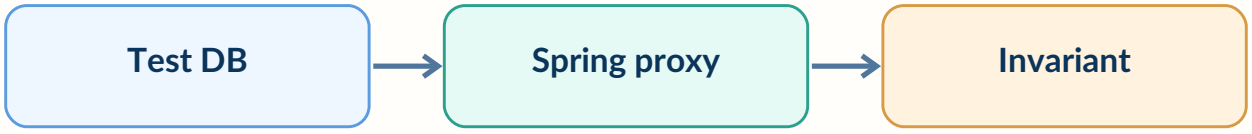
Why this matters: Test ORM mappings, security rules and rollback in managed contexts.

How do you test authentication and authorization paths?

IN ONE CLEAR ANSWER

Cover unauthenticated, authenticated-but-forbidden, permitted and wrong-resource-owner cases. Security tests should include filters and method-level authorization where used.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
@DataJpaTest
class OrderRepositoryTest { /* real SQL */ }
```

INTERVIEW FOLLOW-UP

Does mocking an authentication service prove security works?

No. The security chain, token validation, and object ownership rules need context and integration evidence.

Why this matters: Test ORM mappings, security rules and rollback in managed contexts.

Test strategy and coverage

Use coverage as a map, not a correctness guarantee.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
assertEquals(1, uniqueOrdersForKey("pay-7"));
```

Five distinct questions are followed by five individual answers to deeper interview follow-ups.

What is the testing pyramid?

IN ONE CLEAR ANSWER

It suggests many fast focused tests, fewer component integration tests and a small number of full end-to-end flows. The exact mix depends on risk, not a fixed percentage.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(1, uniqueOrdersForKey("pay-7"));
```

INTERVIEW FOLLOW-UP

Can thousands of mocked tests replace one checkout integration test?

No. Independent mocked collaborators do not establish that payments, database writes and messages work together.

Why this matters: Use coverage as a map, not a correctness guarantee.

What is code coverage and what can it miss?

IN ONE CLEAR ANSWER

Line and branch coverage show which parts of code tests exercised, not whether outcomes were correct. A test can execute a line without asserting an important rule.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(1, uniqueOrdersForKey("pay-7"));
```

INTERVIEW FOLLOW-UP

Is 100 percent coverage evidence of correctness?

No. Missing assertions, faulty test data and untested external failures can remain despite full line coverage.

Why this matters: Use coverage as a map, not a correctness guarantee.

How does mutation testing improve test assessment?

IN ONE CLEAR ANSWER

A mutation tool changes small parts of production code, such as a comparison or return value, and checks whether tests catch the change. Surviving meaningful mutations reveal weak assertions.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(1, uniqueOrdersForKey("pay-7"));
```

INTERVIEW FOLLOW-UP

What should you do with an equivalent mutation?

Document it and avoid treating it as a missing test if it cannot change observable behavior.

Why this matters: Use coverage as a map, not a correctness guarantee.

What causes flaky tests?

IN ONE CLEAR ANSWER

Flakiness comes from shared state, timing assumptions, randomness, environment dependence, unstable network access and nondeterministic ordering. Diagnose and remove the cause rather than hiding it with retries.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(1, uniqueOrdersForKey("pay-7"));
```

INTERVIEW FOLLOW-UP

Are automatic test retries a valid fix?

Not alone. They may reduce noise temporarily but can mask real concurrency or production defects.

Why this matters: Use coverage as a map, not a correctness guarantee.

What is the difference between integration testing and contract testing?

IN ONE CLEAR ANSWER

Integration testing checks concrete collaborators together; contract testing checks that a consumer and provider agree on a defined interface and behavior. Both are useful across service boundaries.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(1, uniqueOrdersForKey("pay-7"));
```

INTERVIEW FOLLOW-UP

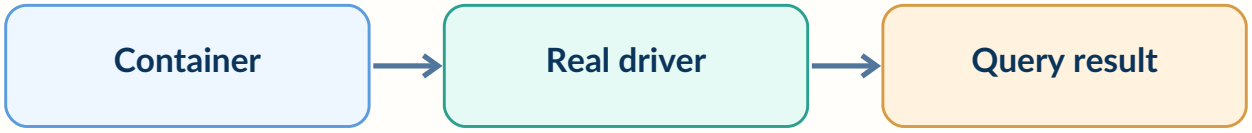
Can consumer contract tests replace production monitoring?

No. They catch compatibility problems in controlled inputs but do not detect all runtime outages and data issues.

Why this matters: Use coverage as a map, not a correctness guarantee.

Integration, containers and regressions

Exercise the actual infrastructure where semantics matter.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
// Example: use PostgreSQL Testcontainers for SQL semantics.
```

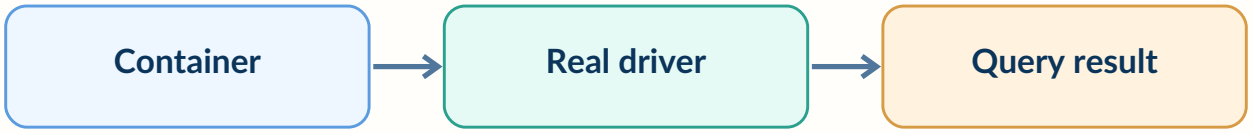
Five distinct questions are followed by five individual answers to deeper interview follow-ups.

When should Testcontainers be used?

IN ONE CLEAR ANSWER

Use Testcontainers to run disposable real dependencies such as PostgreSQL or Kafka for tests that require their behavior. Pin versions, manage cleanup and keep tests repeatable in CI.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
// Example: use PostgreSQL Testcontainers for SQL semantics.
```

INTERVIEW FOLLOW-UP

What is a downside of sharing one container across unrelated tests?

State and order coupling may leak between tests; use isolation strategies or carefully managed independent schemas/data.

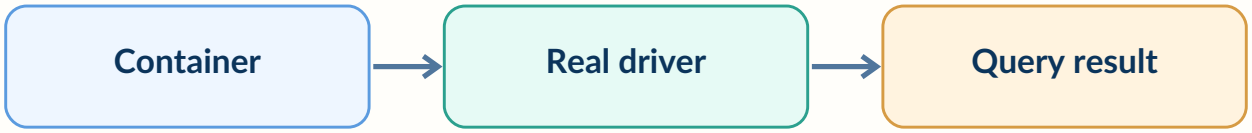
Why this matters: Exercise the actual infrastructure where semantics matter.

How do you test a transaction outbox workflow?

IN ONE CLEAR ANSWER

Persist the business row and outbox row in one transaction, then simulate relay failures, duplicates and replay. Validate idempotent consumer effects with real storage boundaries.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
// Example: use PostgreSQL Testcontainers for SQL semantics.
```

INTERVIEW FOLLOW-UP

What does an outbox unit test fail to prove?

It does not prove durable atomic commit, database constraints, CDC behavior or broker delivery.

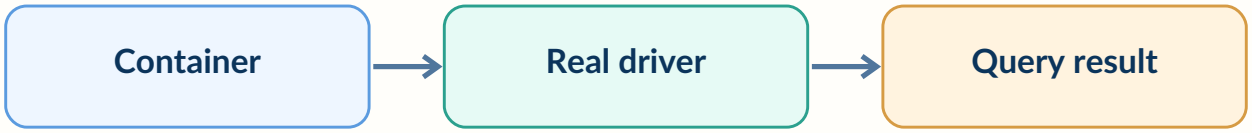
Why this matters: Exercise the actual infrastructure where semantics matter.

How should tests verify cache behavior?

IN ONE CLEAR ANSWER

Test cache hit, miss, invalidation, expiration and stale-load races against explicit freshness requirements. Use real cache integration tests for TTL and serialization semantics.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
// Example: use PostgreSQL Testcontainers for SQL semantics.
```

INTERVIEW FOLLOW-UP

Why is a mock Redis client insufficient for eviction tests?

It has no real eviction algorithm, memory policy, network protocol or persistence semantics.

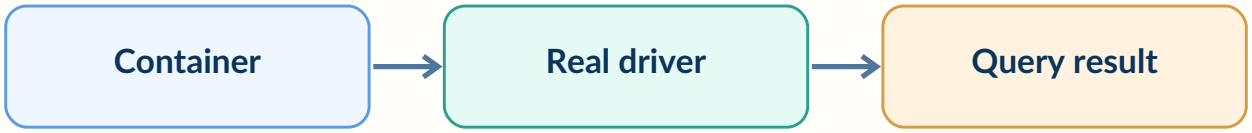
Why this matters: Exercise the actual infrastructure where semantics matter.

How do you test idempotent payment requests?

IN ONE CLEAR ANSWER

Use a stable key in repeated requests; test same-key/same-intent replay, same-key/different-intent conflict and concurrent duplicates. Verify durable storage with integration tests.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
// Example: use PostgreSQL Testcontainers for SQL semantics.
```

INTERVIEW FOLLOW-UP

Why can a local ConcurrentHashMap give false confidence?

It loses state on restart and is not shared across instances, so duplicate charges remain possible.

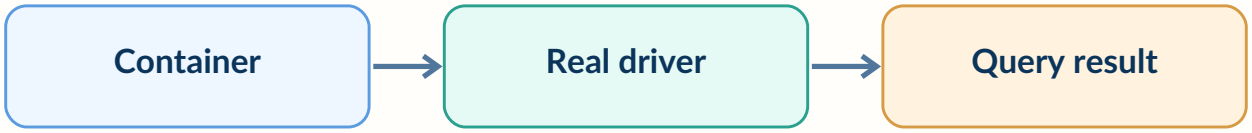
Why this matters: Exercise the actual infrastructure where semantics matter.

How do you structure a high-value test for a bug fix?

IN ONE CLEAR ANSWER

First reproduce the original failure with minimal data, assert the required result, then change the implementation and keep the test as regression coverage.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
// Example: use PostgreSQL Testcontainers for SQL semantics.
```

INTERVIEW FOLLOW-UP

What if the regression test passes before the fix?

It may not reproduce the defect; strengthen the scenario so it fails on the broken implementation.

Why this matters: Exercise the actual infrastructure where semantics matter.

Refactoring and performance tests

Write assertions that survive internal implementation changes.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
assertEquals(expectedState, service.process(command));
```

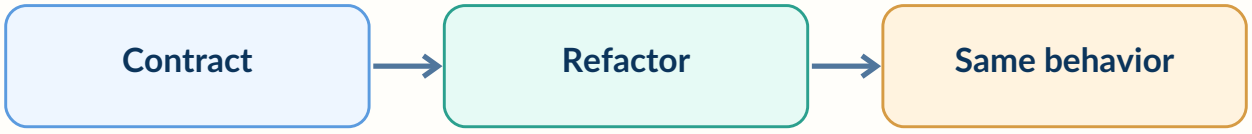
Five distinct questions are followed by five individual answers to deeper interview follow-ups.

How do you prevent tests from becoming brittle during refactoring?

IN ONE CLEAR ANSWER

Assert public behavior, externally important state and required interactions, not private helper calls or incidental ordering. Use builders and focused fixture factories for clarity.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(expectedState, service.process(command));
```

INTERVIEW FOLLOW-UP

What makes a refactor-safe service test?

The test continues to pass when implementation internals change but observable business semantics stay the same.

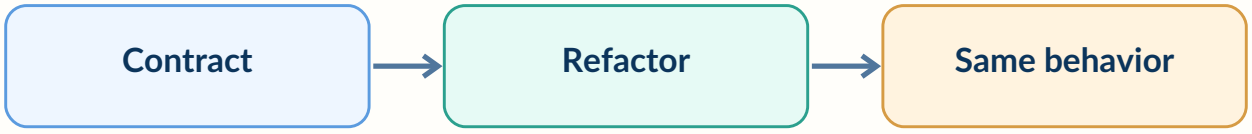
Why this matters: Write assertions that survive internal implementation changes.

How should a senior engineer review a weak test suite?

IN ONE CLEAR ANSWER

Group failures by untested invariants, mocking boundaries, flakiness, slow dependencies and missing negative cases. Add tests where production risk is highest instead of blindly raising assertion counts.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(expectedState, service.process(command));
```

INTERVIEW FOLLOW-UP

What evidence shows the suite is improving?

Lower escaped defects and flakiness, faster actionable feedback, stronger mutation scores and reliable integration failures.

Why this matters: Write assertions that survive internal implementation changes.

How should a team decide what to mock?

IN ONE CLEAR ANSWER

Keep domain decisions real, mock costly or nondeterministic collaborators in unit tests and test high-risk adapters against real systems. Define the behavioral boundary being proved in each test.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(expectedState, service.process(command));
```

INTERVIEW FOLLOW-UP

When is testing against a real service essential?

When transaction, serialization, database query, security filter, broker delivery or protocol behavior is the actual subject of the test.

Why this matters: Write assertions that survive internal implementation changes.

How should you test a feature-flag rollout?

IN ONE CLEAR ANSWER

Cover enabled/disabled behavior, a missing flag, default policy and stale flag state. Assert sensitive behavior is denied safely and avoid embedding remote flag calls in pure logic tests.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(expectedState, service.process(command));
```

INTERVIEW FOLLOW-UP

Why should a feature flag default be tested?

An unavailable configuration service may expose an unsafe capability if the default is not explicit.

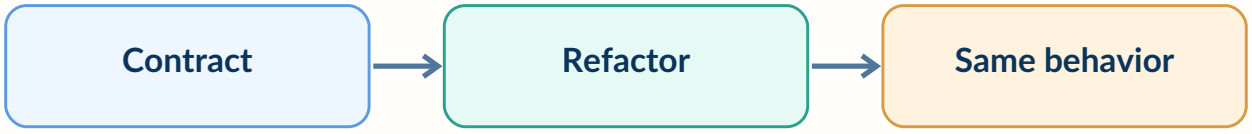
Why this matters: Write assertions that survive internal implementation changes.

How do you test performance-sensitive logic without flaky timing assertions?

IN ONE CLEAR ANSWER

Use focused microbenchmarks for performance, representative load tests for system behavior and functional assertions for correctness. Use time limits only when they express a stable operational contract.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
assertEquals(expectedState, service.process(command));
```

INTERVIEW FOLLOW-UP

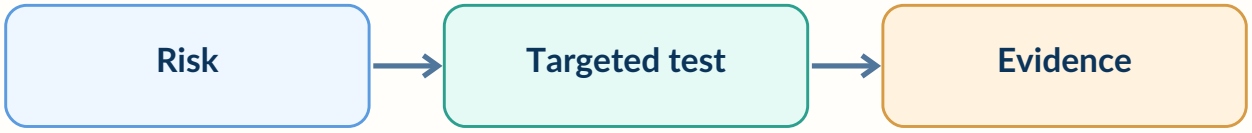
Why is a single short System.nanoTime test weak evidence?

JIT warm-up, GC and machine load can dominate timing; compare distributions under controlled benchmarking conditions.

Why this matters: Write assertions that survive internal implementation changes.

Release readiness

Review neglected failure paths, cleanup and input generation.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
try (var stream = Files.lines(path)) { /* test */ }
```

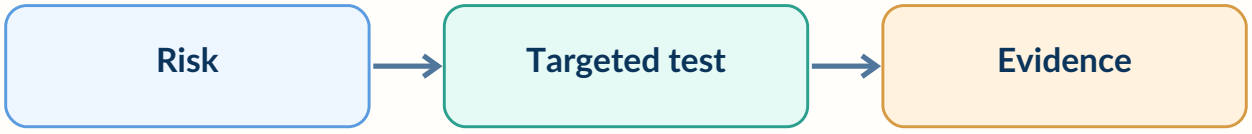
Five distinct questions are followed by five individual answers to deeper interview follow-ups.

What belongs in a test review checklist before release?

IN ONE CLEAR ANSWER

Check business invariants, negative paths, cancellation, repeat events, data isolation, CI reproducibility and meaningful assertions. Record which boundaries remain untested and their operational mitigations.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
try (var stream = Files.lines(path)) { /* test */ }
```

INTERVIEW FOLLOW-UP

How should a team document an untested failure path?

State the scenario, impact, owner, planned test and production detection or rollback approach rather than claiming complete verification.

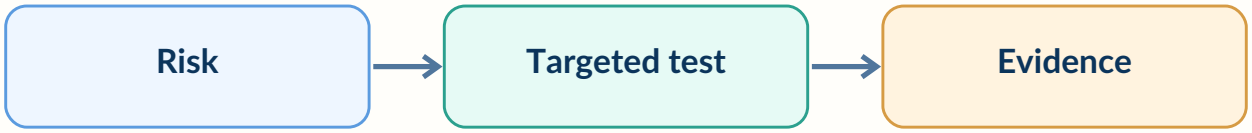
Why this matters: Review neglected failure paths, cleanup and input generation.

How should tests verify resource cleanup?

IN ONE CLEAR ANSWER

Create a controlled resource that records close calls, trigger both success and failure paths, and assert cleanup. For real I/O resources, an integration test should check file/socket ownership and leaks.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
try (var stream = Files.lines(path)) { /* test */ }
```

INTERVIEW FOLLOW-UP

Does try-with-resources always make cleanup successful?

It guarantees close is attempted in scope, but close itself can fail; test suppressed exceptions where they affect the contract.

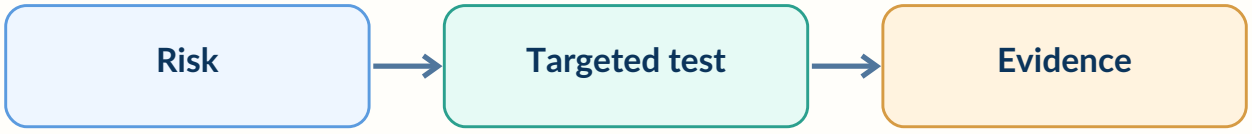
Why this matters: Review neglected failure paths, cleanup and input generation.

How do you test retries against non-idempotent dependencies?

IN ONE CLEAR ANSWER

Use a controlled provider that records stable operation IDs and returns uncertain and definite failures. Assert the client retries only allowed outcomes and reuses idempotency intent.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
try (var stream = Files.lines(path)) { /* test */ }
```

INTERVIEW FOLLOW-UP

Why is blind retry after a timeout unsafe?

The first remote operation may have succeeded even though its response was lost, so a new intent can repeat the effect.

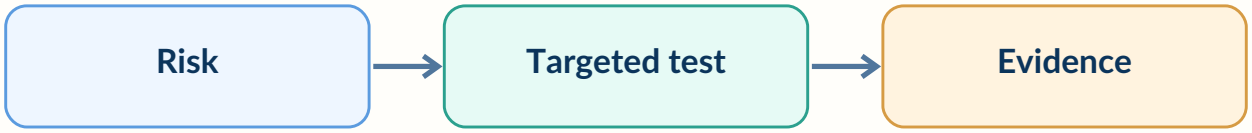
Why this matters: Review neglected failure paths, cleanup and input generation.

How should an API test handle timestamps and time zones?

IN ONE CLEAR ANSWER

Inject Clock for unit tests, use explicit Instant/ZoneId values, and test midnight and daylight-saving boundaries. Avoid LocalDateTime.now() in assertions.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
try (var stream = Files.lines(path)) { /* test */ }
```

INTERVIEW FOLLOW-UP

Why do tests fail around midnight?

Their expected date can be computed at a different instant or zone from the production code; control both.

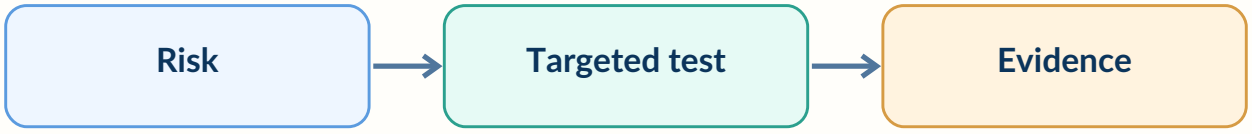
Why this matters: Review neglected failure paths, cleanup and input generation.

How can property-based testing complement examples?

IN ONE CLEAR ANSWER

Generate many domain-valid inputs and check universal rules such as nonnegative balances or encode/decode round trips. Keep targeted examples for known edge cases and regression bugs.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
try (var stream = Files.lines(path)) { /* test */ }
```

INTERVIEW FOLLOW-UP

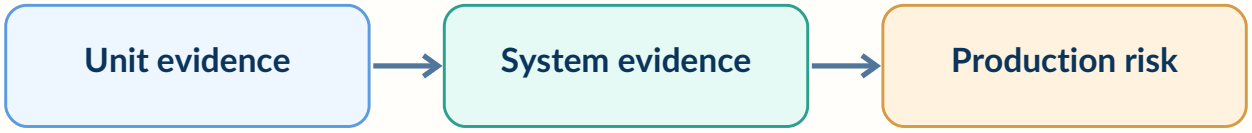
Does random data automatically find race conditions?

No. It explores inputs, not necessarily interleavings, and failures need reproducible seeds and minimized counterexamples.

Why this matters: Review neglected failure paths, cleanup and input generation.

Architect scenarios

Connect test evidence to failure models and operational guarantees.



INTERVIEW FOCUS

Explain what the test proves, what the mock only simulates, and which production guarantee still needs a real integration test.

RELATED SYNTAX

```
verify(eventPublisher, times(1)).publish(event);
```

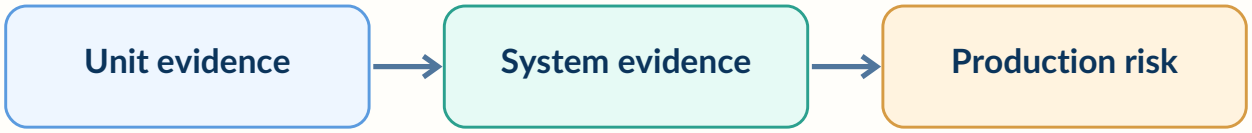
Five distinct questions are followed by five individual answers to deeper interview follow-ups.

What is a snapshot test and when is it useful?

IN ONE CLEAR ANSWER

It compares output to a stored approved representation, often JSON or UI structure. Use it for stable contracts with clear review; large opaque snapshots create noisy approvals.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(eventPublisher, times(1)).publish(event);
```

INTERVIEW FOLLOW-UP

What can cause a harmless-looking snapshot update to be dangerous?

An accepted diff may remove required fields or expose sensitive data if no reviewer checks semantic meaning.

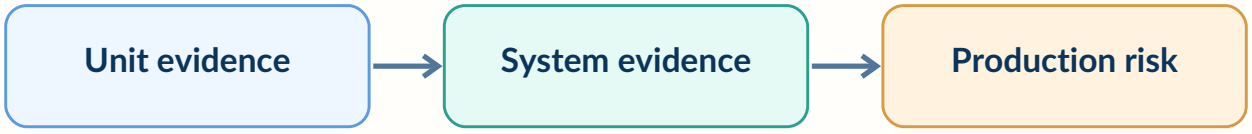
Why this matters: Connect test evidence to failure models and operational guarantees.

How should tests cover retries and circuit breakers together?

IN ONE CLEAR ANSWER

Use a fake clock and failing dependency to force failure thresholds, open/half-open states and bounded retries. Verify the combined call budget so nested retry layers do not amplify traffic.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(eventPublisher, times(1)).publish(event);
```

INTERVIEW FOLLOW-UP

Why can double retries cause overload?

Three retries in a gateway and three in a client can create many attempts for one user request, increasing pressure during an outage.

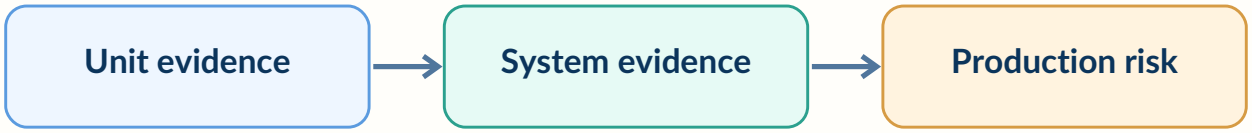
Why this matters: Connect test evidence to failure models and operational guarantees.

How can tests expose tenant-isolation defects?

IN ONE CLEAR ANSWER

Create two tenants with overlapping local IDs, then assert user A cannot query, edit or cache user B's resource. Integration tests should check the actual query predicates and security rules.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(eventPublisher, times(1)).publish(event);
```

INTERVIEW FOLLOW-UP

Why is mocking repository.findById risky in this test?

The mock bypasses tenant filtering in generated SQL and can return preselected safe data even when production code is unsafe.

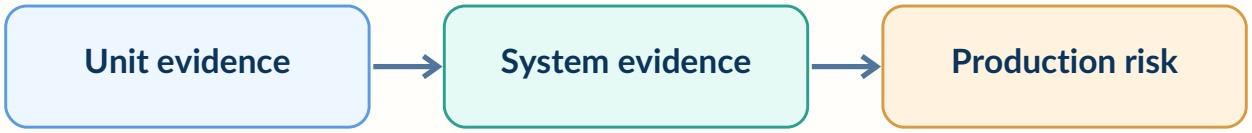
Why this matters: Connect test evidence to failure models and operational guarantees.

How should CI select and report test suites?

IN ONE CLEAR ANSWER

Run fast unit tests early, block merge on essential integration/security checks and publish failed test names, logs and artifacts. Quarantine flaky tests only with an owner and deadline.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(eventPublisher, times(1)).publish(event);
```

INTERVIEW FOLLOW-UP

What should happen when an integration environment is unavailable?

Surface a clear infrastructure failure and decide gate policy explicitly; never silently claim the suite passed.

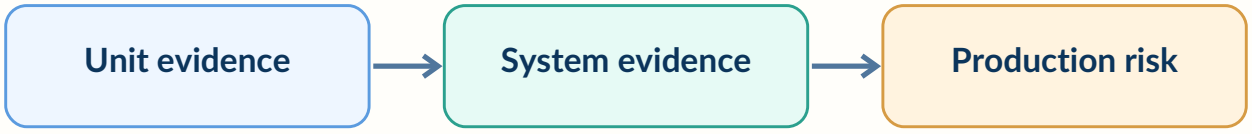
Why this matters: Connect test evidence to failure models and operational guarantees.

How do you explain test confidence in an architect interview?

IN ONE CLEAR ANSWER

Describe the invariant, test boundary, failure injection, evidence, and limitations. Unit tests prove local decisions; integration tests prove concrete collaborators; monitoring validates assumptions after deployment.

VISUAL MEMORY MAP



RELATED JAVA / JUNIT EXAMPLE

```
verify(eventPublisher, times(1)).publish(event);
```

INTERVIEW FOLLOW-UP

Can any test suite guarantee production has no defects?

No. Tests sample behaviors under controlled conditions. Combine them with correct design, reviews, staged releases, monitoring and recovery.

Why this matters: Connect test evidence to failure models and operational guarantees.

HAND-ON CODING

12 practical Java testing labs

Each lab is a complete JUnit Jupiter / Mockito test class. The source ZIP contains a Maven pom.xml, run commands, and dependency versions.

C01	Assertions and money boundaries	**Lab01Test.java**
C02	Parameterized boundary tests	**Lab02Test.java**
C03	Nested fixtures and lifecycle	**Lab03Test.java**
C04	Exceptions and failure contracts	**Lab04Test.java**
C05	Fake clock and time-dependent logic	**Lab05Test.java**
C06	Async futures and controlled completion	**Lab06Test.java**
C07	MockitoExtension with @Mock and @InjectMocks	**Lab07Test.java**
C08	Stubbing failures and argument matchers	**Lab08Test.java**
C09	ArgumentCaptor and event contents	**Lab09Test.java**
C10	Consecutive stubbing and retry count	**Lab10Test.java**
C11	Spy-safe stubbing and partial mocking	**Lab11Test.java**
C12	Interaction order and idempotency	**Lab12Test.java**

The filenames in the project contain the descriptive full names (for example Lab07ServiceMockitoTest.java). Maven automatically discovers test classes ending in Test.

Assertions and money boundaries

YOUR TASK

Goal: Understand a meaningful passing behavior and a failure or boundary case.
Run: `mvn -Dtest=Lab01AssertionsTest test`
Expected: The test class should pass when its dependencies and JDK are configured.

LAB SOURCE PREVIEW: LAB01ASSERTIONSTEST.JAVA

```
package guide;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
class Lab01AssertionsTest {
  static int priceWithTax(int cents, int taxPct) {
    if (cents < 0 || taxPct < 0) throw new IllegalArgumentException("negative");
    return Math.addExact(cents, Math.multiplyExact(cents, taxPct) / 100);
  }
  @Test void returnsCalculatedTotal() {
    int price = priceWithTax(200, 10);
    assertAll("price", () -> assertEquals(220, price), () -> assertTrue(price >= 200));
  }
  @Test void rejectsNegativePrice() {
    assertThrows(IllegalArgumentException.class, () -> priceWithTax(-1, 10));
  }
}
```

REVIEW AND DEEPER QUESTION

What business outcome does this lab actually prove? Which system-level behavior would still require a database, HTTP or broker integration test?

The full uncompacted source, imports and test project are in the downloadable coding ZIP.

Parameterized boundary tests

YOUR TASK

Goal: Understand a meaningful passing behavior and a failure or boundary case.
Run: `mvn -Dtest=Lab02ParametersTest test`
Expected: The test class should pass when its dependencies and JDK are configured.

LAB SOURCE PREVIEW: LAB02PARAMETERSTEST.JAVA

```
package guide;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.NullAndEmptySource;
import static org.junit.jupiter.api.Assertions.*;
class Lab02ParametersTest {
  static boolean eligible(int age) { return age >= 18; }
  @ParameterizedTest @CsvSource({"17,false", "18,true", "19,true", "0,false"})
  void checksAgeBoundary(int age, boolean expected) { assertEquals(expected, eligible(age)); }
  @ParameterizedTest @NullAndEmptySource
  void rejectsMissingCode(String code) { assertFalse(code != null && !code.isBlank()); }
}
```

REVIEW AND DEEPER QUESTION

What business outcome does this lab actually prove? Which system-level behavior would still require a database, HTTP or broker integration test?

The full uncompacted source, imports and test project are in the downloadable coding ZIP.

Nested fixtures and lifecycle

YOUR TASK

Goal: Understand a meaningful passing behavior and a failure or boundary case.
Run: `mvn -Dtest=Lab03LifecycleTest test`
Expected: The test class should pass when its dependencies and JDK are configured.

LAB SOURCE PREVIEW: LAB03LIFECYCLETEST.JAVA

```
package guide;
import org.junit.jupiter.api.*;
import java.util.*;
import static org.junit.jupiter.api.Assertions.*;
class Lab03LifecycleTest {
  private List<String> queue;
  @BeforeEach void setUp() { queue = new ArrayList<>(); }
  @Nested class EmptyQueue {
    @Test void beginsEmpty() { assertTrue(queue.isEmpty()); }
    @Test void addingItemChangesSize() { queue.add("A"); assertEquals(1, queue.size()); }
  }
  @Nested class NonEmptyQueue {
    @BeforeEach void seed() { queue.add("first"); }
    @Test void preservesItem() { assertEquals("first", queue.get(0)); }
  }
}
```

REVIEW AND DEEPER QUESTION

What business outcome does this lab actually prove? Which system-level behavior would still require a database, HTTP or broker integration test?

The full uncompacted source, imports and test project are in the downloadable coding ZIP.

Exceptions and failure contracts

YOUR TASK

Goal: Understand a meaningful passing behavior and a failure or boundary case.
Run: `mvn -Dtest=Lab04ExceptionsTest test`
Expected: The test class should pass when its dependencies and JDK are configured.

LAB SOURCE PREVIEW: LAB04EXCEPTIONSTEST.JAVA

```
package guide;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
class Lab04ExceptionsTest {
  static int divide(int numerator, int denominator) {
    if (denominator == 0) throw new IllegalArgumentException("denominator is zero");
    return numerator / denominator;
  }
  @Test void reportsUsefulError() {
    var failure = assertThrowsExactly(IllegalArgumentException.class, () -> divide(10, 0));
    assertEquals("denominator is zero", failure.getMessage());
  }
  @Test void succeedsForKnownValue() { assertEquals(5, assertDoesNotThrow(() -> divide(10,
2))); }
}
```

REVIEW AND DEEPER QUESTION

What business outcome does this lab actually prove? Which system-level behavior would still require a database, HTTP or broker integration test?

The full uncompacted source, imports and test project are in the downloadable coding ZIP.

Fake clock and time-dependent logic

YOUR TASK

Goal: Understand a meaningful passing behavior and a failure or boundary case.
Run: `mvn -Dtest=Lab05ClockTest test`
Expected: The test class should pass when its dependencies and JDK are configured.

LAB SOURCE PREVIEW: LAB05CLOCKTEST.JAVA

```
package guide;
import org.junit.jupiter.api.Test;
import java.time.*;
import static org.junit.jupiter.api.Assertions.*;
class Lab05ClockTest {
  static boolean expired(Instant deadline, Clock clock) { return !
clock.instant().isBefore(deadline); }
  @Test void checksExactBoundaryWithoutSleeping() {
    Clock fixed = Clock.fixed(Instant.parse("2026-01-01T00:00:00Z"), ZoneOffset.UTC);
    assertFalse(expired(Instant.parse("2026-01-01T00:00:01Z"), fixed));
    assertTrue(expired(Instant.parse("2026-01-01T00:00:00Z"), fixed));
  }
}
```

REVIEW AND DEEPER QUESTION

What business outcome does this lab actually prove? Which system-level behavior would still require a database, HTTP or broker integration test?

The full uncompacted source, imports and test project are in the downloadable coding ZIP.

Async futures and controlled completion

YOUR TASK

Goal: Understand a meaningful passing behavior and a failure or boundary case.
Run: `mvn -Dtest=Lab06AsyncTest test`
Expected: The test class should pass when its dependencies and JDK are configured.

LAB SOURCE PREVIEW: LAB06ASYNCTEST.JAVA

```
package guide;
import org.junit.jupiter.api.Test;
import java.util.concurrent.*;
import java.time.Duration;
import static org.junit.jupiter.api.Assertions.*;
class Lab06AsyncTest {
  static CompletableFuture<Integer> doubleLater(CompletableFuture<Integer> source) {
    return source.thenApply(n -> n * 2);
  }
  @Test void verifiesControlledCompletion() {
    CompletableFuture<Integer> source = new CompletableFuture<>();
    var output = doubleLater(source); source.complete(12);
    assertTimeout(Duration.ofSeconds(1), () -> assertEquals(24, output.get(1,
TimeUnit.SECONDS)));
  }
  @Test void preservesFailure() {
    var source = new CompletableFuture<Integer>();
    var output = doubleLater(source); source.completeExceptionally(new
IllegalStateException("offline"));
    assertThrows(CompletionException.class, output::join);
  }
}
```

REVIEW AND DEEPER QUESTION

What business outcome does this lab actually prove? Which system-level behavior would still require a database, HTTP or broker integration test?

The full uncompacted source, imports and test project are in the downloadable coding ZIP.

MockitoExtension with @Mock and @InjectMocks

YOUR TASK

Goal: Understand a meaningful passing behavior and a failure or boundary case.
Run: `mvn -Dtest=Lab07ServiceMockitoTest test`
Expected: The test class should pass when its dependencies and JDK are configured.

LAB SOURCE PREVIEW: LAB07SERVICEMOCKITOTEST.JAVA

```
package guide;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;
@ExtendWith(MockitoExtension.class)
class Lab07ServiceMockitoTest {
  interface UserRepository { String findName(long id); }
  static class UserService {
    private final UserRepository repo;
    UserService(UserRepository repo) { this.repo = repo; }
    String displayName(long id) { return repo.findName(id).toUpperCase(); }
  }
  @Mock UserRepository repo;
  @InjectMocks UserService service;
  @Test void returnsUppercaseName() {
    when(repo.findName(7L)).thenReturn("meena");
    assertEquals("MEENA", service.displayName(7L));
    verify(repo).findName(7L);
  }
}
```

REVIEW AND DEEPER QUESTION

What business outcome does this lab actually prove? Which system-level behavior would still require a database, HTTP or broker integration test?

The full uncompacted source, imports and test project are in the downloadable coding ZIP.

Stubbing failures and argument matchers

YOUR TASK

Goal: Understand a meaningful passing behavior and a failure or boundary case.
Run: `mvn -Dtest=Lab08StubbingTest test`
Expected: The test class should pass when its dependencies and JDK are configured.

LAB SOURCE PREVIEW: LAB08STUBBINGTEST.JAVA

```
package guide;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;
import static org.mockito.ArgumentMatchers.*;
@ExtendWith(MockitoExtension.class)
class Lab08StubbingTest {
  interface Gateway { String pay(String key, int cents); }
  static class PaymentService {
    private final Gateway gateway;
    PaymentService(Gateway gateway) { this.gateway = gateway; }
    String charge(String key, int cents) { if (cents <= 0) throw new
IllegalArgumentException(); return gateway.pay(key, cents); }
  }
  @Mock Gateway gateway;
  @Test void returnsProviderReceipt() {
    when(gateway.pay(eq("pay-7"), anyInt())).thenReturn("receipt-7");
    assertEquals("receipt-7", new PaymentService(gateway).charge("pay-7", 300));
    verify(gateway).pay("pay-7", 300);
  }
  @Test void rejectsBadAmountWithoutCallingProvider() {
    assertThrows(IllegalArgumentException.class, () -> new PaymentService(gateway).charge("x",
0));
    verifyNoInteractions(gateway);
  }
}
```

REVIEW AND DEEPER QUESTION

What business outcome does this lab actually prove? Which system-level behavior would still require a database, HTTP or broker integration test?

The full uncompacted source, imports and test project are in the downloadable coding ZIP.

ArgumentCaptor and event contents

YOUR TASK

Goal: Understand a meaningful passing behavior and a failure or boundary case.
Run: `mvn -Dtest=Lab09CaptorTest test`
Expected: The test class should pass when its dependencies and JDK are configured.

LAB SOURCE PREVIEW: LAB09CAPTORTEST.JAVA

```
package guide;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.ArgumentCaptor;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;
@ExtendWith(MockitoExtension.class)
class Lab09CaptorTest {
  record OrderEvent(String orderId, int totalCents) {}
  interface Publisher { void publish(OrderEvent event); }
  static class Checkout {
    private final Publisher publisher;
    Checkout(Publisher publisher) { this.publisher = publisher; }
    void submit(String id, int cents) { publisher.publish(new OrderEvent(id, cents)); }
  }
  @Mock Publisher publisher;
  @Test void publishesTheCorrectBusinessData() {
    new Checkout(publisher).submit("ord-5", 450);
    var captor = ArgumentCaptor.forClass(OrderEvent.class);
    verify(publisher).publish(captor.capture());
    assertAll(() -> assertEquals("ord-5", captor.getValue().orderId()),
              () -> assertEquals(450, captor.getValue().totalCents()));
  }
}
```

REVIEW AND DEEPER QUESTION

What business outcome does this lab actually prove? Which system-level behavior would still require a database, HTTP or broker integration test?

The full uncompacted source, imports and test project are in the downloadable coding ZIP.

Consecutive stubbing and retry count

YOUR TASK

Goal: Understand a meaningful passing behavior and a failure or boundary case.
Run: `mvn -Dtest=Lab10RetryTest test`
Expected: The test class should pass when its dependencies and JDK are configured.

LAB SOURCE PREVIEW: LAB10RETRYTEST.JAVA

```
package guide;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;
@ExtendWith(MockitoExtension.class)
class Lab10RetryTest {
  interface Worker { boolean attempt(); }
  static class Retrier {
    private final Worker worker;
    Retrier(Worker worker) { this.worker = worker; }
    boolean run(int maxAttempts) {
      for (int n = 0; n < maxAttempts; n++) if (worker.attempt()) return true;
      return false;
    }
  }
  @Mock Worker worker;
  @Test void stopsAfterSuccess() {
    when(worker.attempt()).thenReturn(false, false, true);
    assertTrue(new Retrier(worker).run(4));
    verify(worker, times(3)).attempt();
  }
}
```

REVIEW AND DEEPER QUESTION

What business outcome does this lab actually prove? Which system-level behavior would still require a database, HTTP or broker integration test?

The full uncompacted source, imports and test project are in the downloadable coding ZIP.

Spy-safe stubbing and partial mocking

YOUR TASK

Goal: Understand a meaningful passing behavior and a failure or boundary case.
Run: `mvn -Dtest=Lab11SpyTest test`
Expected: The test class should pass when its dependencies and JDK are configured.

LAB SOURCE PREVIEW: LAB11SPYTEST.JAVA

```
package guide;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;
class Lab11SpyTest {
  static class Formatter {
    String normalize(String text) { if (text == null) throw new
IllegalArgumentException("null"); return text.trim(); }
    String greeting(String name) { return "Hi " + normalize(name); }
  }
  @Test void doReturnPreventsRealMethodDuringStubbing() {
    Formatter spy = spy(new Formatter());
    doReturn("Meena").when(spy).normalize(null);
    assertEquals("Hi Meena", spy.greeting(null));
    verify(spy).normalize(null);
  }
  @Test void realClassStillRejectsNull() {
    assertThrows(IllegalArgumentException.class, () -> new Formatter().normalize(null));
  }
}
```

REVIEW AND DEEPER QUESTION

What business outcome does this lab actually prove? Which system-level behavior would still require a database, HTTP or broker integration test?

The full uncompacted source, imports and test project are in the downloadable coding ZIP.

Interaction order and idempotency

YOUR TASK

Goal: Understand a meaningful passing behavior and a failure or boundary case.
Run: `mvn -Dtest=Lab12OrderAndIdempotencyTest test`
Expected: The test class should pass when its dependencies and JDK are configured.

LAB SOURCE PREVIEW: LAB12ORDERANDIDEMPOTENCYTEST.JAVA

```
package guide;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.InOrder;
import org.mockito.junit.jupiter.MockitoExtension;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;
@ExtendWith(MockitoExtension.class)
class Lab12OrderAndIdempotencyTest {
  interface Ledger { boolean createIfAbsent(String key); }
  interface Sender { void send(String key); }
  static class Workflow {
    private final Ledger ledger; private final Sender sender;
    Workflow(Ledger ledger, Sender sender) { this.ledger = ledger; this.sender = sender; }
    boolean process(String key) { if (!ledger.createIfAbsent(key)) return false;
sender.send(key); return true; }
  }
  @Mock Ledger ledger; @Mock Sender sender;
  @Test void sendsOnlyAfterAcceptedLocalIntent() {
    when(ledger.createIfAbsent("request-1")).thenReturn(true, false);
    Workflow work = new Workflow(ledger, sender);
    assertTrue(work.process("request-1")); assertFalse(work.process("request-1"));
    InOrder order = inOrder(ledger, sender);
    order.verify(ledger).createIfAbsent("request-1");
    order.verify(sender).send("request-1");
    order.verify(ledger).createIfAbsent("request-1");
    verify(sender, times(1)).send("request-1");
  }
}
```

REVIEW AND DEEPER QUESTION

What business outcome does this lab actually prove? Which system-level behavior would still require a database, HTTP or broker integration test?

The full uncompacted source, imports and test project are in the downloadable coding ZIP.

30-DAY PRACTICE

30-day interview preparation plan

DAY 01	Testing fundamentals - Q001 to Q005
DAY 02	JUnit annotations and assertions - Q006 to Q010
DAY 03	Fixtures and lifecycle - Q011 to Q015
DAY 04	Parameterized testing - Q016 to Q020
DAY 05	Errors, futures and time - Q021 to Q025
DAY 06	JUnit Platform and extensions - Q026 to Q030
DAY 07	Mockito object setup - Q031 to Q035
DAY 08	Mockito stubbing patterns - Q036 to Q040
DAY 09	Matchers and captors - Q041 to Q045
DAY 10	Behavior verification - Q046 to Q050
DAY 11	Spies and special cases - Q051 to Q055
DAY 12	Strictness and test isolation - Q056 to Q060
DAY 13	Fakes, contracts and concurrency - Q061 to Q065
DAY 14	Async and Spring MVC - Q066 to Q070
DAY 15	Spring Data and security - Q071 to Q075

30-DAY PRACTICE

30-day interview preparation plan

DAY 16	Test strategy and coverage - Q076 to Q080
DAY 17	Integration, containers and regressions - Q081 to Q085
DAY 18	Refactoring and performance tests - Q086 to Q090
DAY 19	Release readiness - Q091 to Q095
DAY 20	Architect scenarios - Q096 to Q100
DAY 21	Implement Lab C01 and Lab C02; explain a negative case
DAY 22	Implement Lab C03 and Lab C04; explain a negative case
DAY 23	Implement Lab C05 and Lab C06; explain a negative case
DAY 24	Implement Lab C07 and Lab C08; explain a negative case
DAY 25	Implement Lab C09 and Lab C10; explain a negative case
DAY 26	Implement Lab C11 and Lab C12; explain a negative case
DAY 27	Mock interview: JUnit annotations, assertions and test design
DAY 28	Mock interview: Mockito, verification and mock pitfalls
DAY 29	System test review: real database, async retries and security
DAY 30	Final revision: retest weak cases and present risk-based evidence

QUICK REVISION

Choose the correct test

USE PLAIN JUNIT + MOCKITO

For pure Java services whose dependencies can be controlled; assert return values and required side effects.

USE A REAL DATABASE TEST

For JPQL/SQL, mapping, unique constraints, transaction isolation and optimistic locking.

USE SPRING CONTEXT OR MVC TESTS

For component wiring, validation, filters, security rules and serialization contracts.

USE A FULL END-TO-END TEST

For a small number of critical user workflows that cross deployed services.

REVIEW THESE PITFALLS

Overmocking; no assertions; shared mutable data; Thread.sleep; unbounded futures; unchecked Mockito defaults; verifying private call chains; assuming one mock call proves a commit.

SOURCES AND RIGHTS

Versions, references and publication notes

OFFICIAL LEARNING REFERENCES

JUnit Jupiter user guide: https://docs.junit.org/5.13.4/user-guide/
Mockito official documentation: https://site.mockito.org/
Maven Surefire test runner: https://maven.apache.org/surefire/

VERSIONED LAB DEPENDENCIES

Java 17, JUnit Jupiter 5.13.4, Mockito JUnit Jupiter 5.24.0, Maven Surefire 3.5.4.

TEST LIMITATIONS

The packaged test classes have not been executed with JUnit and Mockito dependencies in the available environment. Before sale or production use, run mvn test on a compatible machine and inspect failures.

This is an original interview preparation guide. Java, JUnit and Mockito are trademarks of their respective owners. All example code is original educational material and is not an official certification or vendor publication.